Week 10 — Linux Processes and Signals

Format: seven daily sections, each a mechanism lesson with one bounded wrapper in `projects/linux-system-programming`

Suggested workload: approximately 21 focused hours, adjustable to mastery

> A program file is bytes. A process is a kernel task that owns an address space, a file table, credentials, and a place in a parent/child tree. This week follows that task from creation through image replacement, death, signals, and the names a shell needs so a terminal can interrupt a job. The last day asks `/proc`, strace, and GDB to show the same objects, and compares them with ToyOS and xv6.

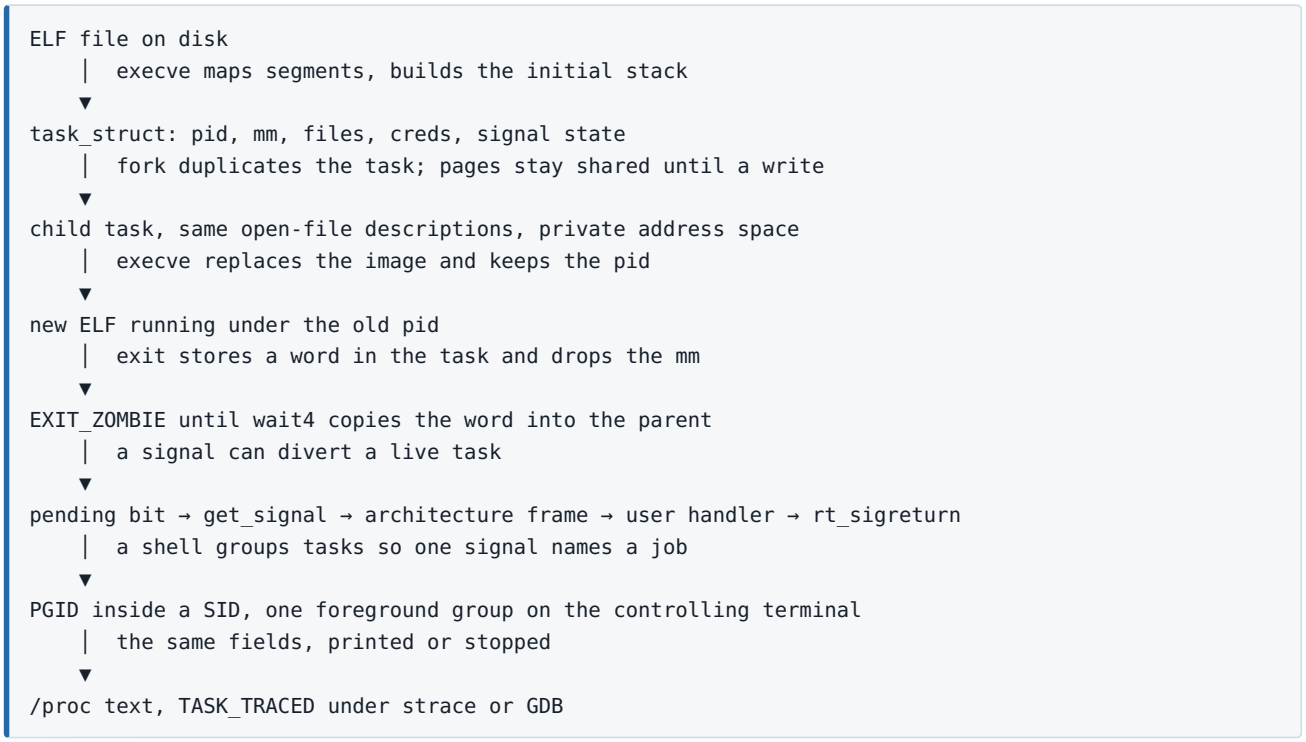
The excerpts in Days 4–7 come from the local kernel tree `c9b2385d6793`, a 5.10.134-based source. Function names, call order, and line numbers move between releases. Record `uname -r` beside that id before treating a line as true of the machine you are running. The POSIX and man-page contracts are what a C program may rely on. A function body is one implementation of that contract, in one tree. Mastery stays open until you produce the evidence yourself. These notes leave completion, confidence, and hours blank.

Read the small kernel before the large one

Use the shared Linux 0.12 → Linux 5.10 map before the detailed day sections. Linux 0.12 places fork, exec, exit, wait, signals, and descriptor lifetime in short paths that can be read end to end. Then use this week's 5.10 excerpts to replace historical segmentation, `a.out`, fixed task arrays, and uniprocessor assumptions with the modern implementation. The 0.12 source is the explanatory map; the 5.10/current source remains the development reference.

Why the seven days are one mechanism

Week 1 ended at exceptions and privilege. Weeks 2 and 3 gave that stream a C abstract machine and an ELF image. Weeks 6–9 built one ToyOS address space, an `iretq` back into it, one user syscall path, and a small virtual filesystem. Linux gives each task its own address space and keeps a kernel record after that space is gone, because another task may still need the exit word.



Days 1–3 establish the live task. Days 4–7 follow it after that picture gets interesting: the exit word, the asynchronous entry, the job name, and the tools that read the kernel objects back.

Day 1 — the task is more than the file

sections/day-001-process-mental-model.md

An ELF file has no instruction pointer and no credentials. A process is the kernel's `task_struct` plus the memory descriptor, file table, signal state, and credentials it references. The pid is a name in a pid namespace. `/proc/self` is a generated view of that task.

Day 2 — fork copies a context and shares the pages until a write

sections/day-002-fork-cow-and-lineage.md

`fork` creates a second task. Descriptor tables are separate; the open file descriptions they point at are shared, so an offset can move under both processes. User pages are copy-on-write. The parent/child link this creates is the link Day 4's `wait` and reparenting walk.

Day 3 — exec replaces the image and keeps the identity

sections/day-003-exec-elf-and-inheritance.md

`execve` unmaps the old image, maps the new ELF, builds a new user stack, and starts at the new entry. The pid stays. Close-on-exec descriptors disappear across that boundary. Signal dispositions reset according to the exec rules. The exit word Day 4 stores describes this image, which may be a program the parent did not originally exec.

Day 4 — the exit word lives in the task

sections/day-004-wait-zombies-and-reparenting.md

`do_exit` stores `exit_code`, drops the address space, and `exit_notify` sets `EXIT_ZOMBIE` until a parent collects the word through `wait4`. The word is kernel state. The program that produced it has already been unmapped. If the parent exits first, `forget_original_parent`

gives the child a new `real_parent`: another thread, a child subreaper, or the namespace init. The lab adopts a grandchild with `PR_SET_CHILD_SUBREAPER` so the new parent is the lab process.

Day 5 — a handler is user code entered asynchronously

sections/day-005-signals-masks-and-safe-handlers.md

`__send_signal` posts a pending bit. A blocked signal stays pending. `get_signal`, on the way back to user mode, dequeues an unblocked signal and either runs a handler, ignores it, or takes the default action. The handler is ordinary user code, so it may call only functions that cannot touch inconsistent libc state. The self-pipe moves the event onto the program's normal path. The frame that makes the handler look like a call, and the `rt_sigreturn` that resumes the interrupted registers, are built by architecture code. x86-64, AArch64, and RISC-V each have their own.

Day 6 — a job is a process group inside a session

sections/day-006-shell-process-groups-and-sessions.md

A shell is a loop of `fork`, `exec`, and `wait`, plus group identity so the terminal can signal every member of a pipeline. `setpgid` moves a task between groups. `setsid` points the session id and the group id at the caller's pid and clears the controlling terminal. One group is the foreground group; keyboard signals go there. Process groups are kernel policy, the same idea on every ISA. The lab records ids and one child group leader. An interactive shell, `setsid`, and `tcsetpgrp` stay off the terminal you are using.

Day 7 — read the objects back, and compare

sections/day-007-proc-strace-gdb-comparison-report.md

`proc_pid_status` prints `task_struct` fields when you read the file. `Z` is `EXIT_ZOMBIE`. A ptrace stop is `TASK_TRACED`. strace is a tracer that prints syscalls from that stop. GDB is the same stop plus ELF symbols, so a breakpoint at `main` is a symbol address after startup has already run. ToyOS has one address space and an explicit marker. xv6 has fork, exec, and wait in a small process table. This Linux tree has the task, the exit word, the signal queues, the session, and the generated `/proc` text. The report keeps those three apart, and it records `uname -r` before any line number.

How the lab stays inside one process family

The project is an offline C17/POSIX lab. Wrappers in `challenges/` are how you run it. Each one resolves the canonical project, builds through that project's Makefile, and checks a marker. They create only descendants of the invoking process, signal only that process or those descendants, reap every child, and run without root or a network.

The orphan exercise uses `PR_SET_CHILD_SUBREAPER` so the new parent is the lab process. The zombie is polled briefly and then waited for. The shell day reads the current session and creates one child group. It leaves the terminal's foreground group where the launcher put it. PID values and `/proc` text vary by run. The stable evidence is a decoded status, a state letter, an expected `errno`, a program marker, or an explicit skip when strace or GDB is absent.

What a finished week can answer

After the seven sections and their wrappers, you should be able to answer, with a source or a run attached to each sentence:

1. Which state is copied, shared, reset, or replaced across `fork` and `exec`?
2. Which object owns a descriptor's offset, and when does close-on-exec act?
3. Why does the kernel keep a dead task until a parent collects the status, and who is that parent after reparenting?
4. Where is a signal while it is blocked, pending, selected, delivered, and returned from?
5. How do session, process group, and terminal foreground differ?
6. Which sentence came from source, ELF metadata, `/proc`, a trace, a debugger stop, or a checked assertion?
7. What is still unproved because of timing, namespaces, a missing tool, kernel version, or architecture?

Write the answers in the Day 7 report. A `WEEK10 PASS` line means the checked invariants held on that run. The report is how you show the mechanism.

Day 1 — A Process Is a Live Task

Target time: approximately 3 hours

- Why a task object exists, beside the ELF file: 35 minutes
- The `task_struct` graph and the objects it references: 50 minutes
- PID namespaces, registers, and page tables: 40 minutes
- One execution trace of `proc_lab`: 30 minutes
- Mastery, then the wrapper as evidence: 25 minutes

> The same ELF file has been started twice. Why does each run have its own instruction pointer, its own page-table root, and its own PID?

Week 2 taught the architectural register file: `RIP` names the next instruction, and a general-purpose register holds a value only while some saved context says it belongs to this stream of execution. Week 5 taught the page-table walk: a virtual address becomes a physical frame only when a root pointer and a chain of table entries say so. Week 3 taught that a C program on disk is an ELF image — `PT_LOAD` segments, `e_entry`, and, for a dynamic executable, `PT_INTERP` — plus an initial stack the loader builds before `main`. Weeks 6–9 built the small operating system that uses those pieces: one bootstrap address space, then kernel tasks with saved registers and private stacks, then a ring-3 process with its own page-table root, entered by `iretq`, calling the kernel through `int 0x80`, with `copy_from_user` checking every user pointer.

Linux is that shape with sharing, credentials, and lifetime added. A program is still an ELF file. A process is a live task plus the kernel objects the task references.

The source excerpts below are from the local tree `5.10.134-15_rc1-10333-gc9b2385d6793` (commit `c9b2385d6793`). That tree is 5.10-based. Field layout in a later mainline release can move. The names quoted here are what this revision actually contains.

---

1. The hardware already has registers and a page-table root

A core can execute one instruction stream at a time. `RIP` (or the program counter on another ISA) selects the instruction. The page-table root selects which translations that instruction's loads and stores use. When the core stops one stream and resumes another, both of those values have to be stored somewhere that outlives the core's architectural registers.

Week 6 installed one kernel address space at bootstrap and left the core in it. Week 7 needed a second fact: two kernel tasks share that address space and still need different resume points. Each ToyOS kernel task therefore keeps a saved context — instruction pointer, stack pointer, callee-saved registers — and a private kernel stack. Switching tasks writes the core's registers into the outgoing context and loads the incoming one. The page tables stay the bootstrap tables, because those tasks are both ring 0 and both trusted with the whole kernel map.

Week 8 added a user image that must be untrusted. `struct process` in `projects/toyos/src/process.c` holds a page-table root, an ELF entry, a user stack, a kernel stack, and a PID. `process_init` builds a private PML4, maps the embedded ELF's `PT_LOAD` segments, and maps a user stack. `process_start` activates that root and enters CPL 3 through `iretq`, which loads user `RIP`, `CS`, `RFLAGS`, `RSP`, and `SS` from a frame the kernel prepared. A later `int 0x80` is the privilege boundary: the CPU switches to the kernel stack, the stub saves the user registers, C handles the request, and `iretq` restores the user continuation. `copy_from_user` walks the user root and copies one checked byte at a time, because a value in a user register is an address in that user map. The kernel copies through the checked path before it uses the bytes.

That is already a process in miniature: identity, a register resume point, a page-table root, and a kernel stack for crossings. It is one process, with one address space, and with authority implied by "this is the user program we loaded." Linux keeps every one of those roles and then allows many tasks to share some of the heavy objects, to carry credentials as data, and to outlive their user mappings long enough for a parent to observe them.

---

2. A program is an ELF file

An ELF file is a sequence of bytes with a header. Week 3 separated two views of those bytes. Section headers describe how the linker grouped `.text`, `.rodata`, `.data`, and `.bss`. Program headers describe the memory image a loader builds. A `PT_LOAD` header supplies a file offset, a virtual address, a file size, a memory size, and permission flags. Bytes past `p_filesz` and inside `p_memsz` are the zero-filled part of the image. `e_entry` is a virtual address in that image. `PT_INTERP`, when present, names another ELF file, the interpreter, whose entry runs before the executable's entry.

None of those fields is an architectural register. The file has no current `RIP`, no open descriptors, no credentials, and no pending signals. Copying the file copies the recipe. Starting it twice requires two live contexts that can each hold a `RIP` and a page-table root.

ToyOS makes the same cut inside one function. `load_elf` checks the magic, ELF64, little-endian, machine 62 (`EM_X86_64`), and each program header of type `ELF_LOAD`. For each such header it allocates frames and calls `map_page_in` on `process->root`. The entry virtual address is stored in `process->entry`. The CPU fetches from that address only after `iretq` has loaded it into `RIP` and the PML4 has been activated. Until then the ELF image is input data.

Linux's loader, visited on Day 3, follows the same headers — `PT_LOAD`, `PT_INTERP`, `e_entry` — and writes them into a new memory descriptor. The descriptor is then hung off an existing task. The file remains the file.

```
ELF file on disk                    live task
────────────────                    ──────────────────────────────
e_entry, PT_LOAD, PT_INTERP         task_struct
bytes, no registers                 saved user registers / kernel stack
no page tables                      mm->pgd  (page-table root)
no PID                              pid, tgid, looked up in a namespace
no credentials                      real_cred, cred
no descriptors                      files, fs
```

---

3. `task_struct` is the schedulable object

In `include/linux/sched.h`, `struct task_struct`, tree `5.10.134-15_rc1-10333-gc9b2385d6793`:

```
struct task_struct {
#ifdef CONFIG_THREAD_INFO_IN_TASK
    /*
     * For reasons of header soup (see current_thread_info()), this
     * must be the first element of task_struct.
     */
    struct thread_info      thread_info;
#endif
    /* -1 unrunnable, 0 runnable, >0 stopped: */
    volatile long           state;

    /*
     * This begins the randomizable portion of task_struct. Only
     * scheduling-critical items should be added above here.
     */
    randomized_struct_fields_start

    void                *stack;
    refcount_t          usage;
    /* Per task flags (PF_*), defined further below: */
    unsigned int            flags;
    unsigned int            ptrace;

#ifdef CONFIG_SMP
    int             on_cpu;
    struct __call_single_node   wake_entry;
#ifdef CONFIG_THREAD_INFO_IN_TASK
    /* Current CPU: */
    unsigned int            cpu;
#endif
```

`state` is the scheduler's classification of this task: runnable, stopped, or unrunnable. `stack` is the kernel stack. That is the stack Week 8's syscall frame lived on. Entry from user mode saves the user registers there; the C syscall handler sees a frame; the return path restores them. `usage` is a reference count on the task object itself, so the object can remain reachable after the task has stopped running. That is the lifetime hook Day 4 will use when a parent still needs an exit status. `current` on a CPU names the `task_struct` that owns the core, the way ToyOS's `current` names the running kernel task.

The rest of the process is a graph of pointers out of this object. The pointers are how sharing works: two tasks can hold the same address, and a reference count on the target decides when the target is freed.

```
task_struct
├── stack, thread          registers and the kernel stack (per task)
├── pid, tgid              numbers cached from struct pid
├── real_parent, parent    lineage
├── mm, active_mm          user address space; page-table root is mm->pgd
├── files                  descriptor table
├── fs                     cwd, root, umask
├── real_cred, cred        credential objects
├── signal, sighand        process-wide signal state and dispositions
├── blocked, pending       this task's mask and pending set
├── nsproxy                namespaces, including the PID namespace
└── exit_state             life after the task has stopped executing
```

---

4. Memory, exit, identity, credentials, files, signals

Still in `include/linux/sched.h`, `struct task_struct`, same tree. The memory and exit fields:

```
    struct mm_struct        *mm;
    struct mm_struct        *active_mm;

    /* Per-thread vma caching: */
    struct vmacache         vmacache;

#ifdef SPLIT_RSS_COUNTING
    struct task_rss_stat        rss_stat;
#endif
    int             exit_state;
    int             exit_code;
    int             exit_signal;
    /* The signal sent when the parent dies: */
    int             pdeath_signal;
```

`mm` is the user address space. `active_mm` is the address space whose page tables are installed while this task runs. A user task points both fields at its own `mm`. A kernel thread has no user image: `copy_mm` in `kernel/fork.c` returns early when `current->mm` is absent, and the thread runs with a borrowed `active_mm` so the core still has a page-table root for kernel mappings. Week 7's kernel tasks were that case in a smaller kernel. They shared the bootstrap page tables and never owned a private user `mm`.

`exit_state`, `exit_code`, and `exit_signal` sit on the task because the user mappings and the "someone must observe how this ended" fact have different lifetimes. The address space can be torn down while the task record still holds a status for the parent. Day 4 follows that handoff. The field is already visible here so the process model includes death as data on the task object.

Identity and lineage, same struct:

```
    pid_t               pid;
    pid_t               tgid;

#ifdef CONFIG_STACKPROTECTOR
    /* Canary value for the -fstack-protector GCC feature: */
    unsigned long           stack_canary;
#endif
    /*
     * Pointers to the (original) parent process, youngest child, younger sibling,
     * older sibling, respectively.  (p->father can be replaced with
     * p->real_parent->pid)
     */

    /* Real parent process: */
    struct task_struct __rcu    *real_parent;

    /* Recipient of SIGCHLD, wait4() reports: */
    struct task_struct __rcu    *parent;

    /*
     * Children/sibling form the list of natural children:
     */
    struct list_head        children;
    struct list_head        sibling;
    struct task_struct      *group_leader;
```

`pid` and `tgid` are `pid_t` values cached on the task. `real_parent` is the creating parent. `parent` is the task that receives `SIGCHLD` and that `wait4` reports. For an ordinary child they name the same task. The structure keeps both because a tracer can become `parent` while `real_parent` remains the creator. `children` and `sibling` are the lists that make "lineage" a walk of kernel pointers. `group_leader` points at the task that represents the thread group. A single-threaded program is its own leader.

Credentials, same struct. The comments are the kernel's own names for the two pointers:

```
    /* MM fault and swap info: this can arguably be seen as either mm-specific or thread-specific:
*/
    unsigned long           min_flt;
    unsigned long           maj_flt;

    /* Empty if CONFIG_POSIX_CPUTIMERS=n */
    struct posix_cputimers      posix_cputimers;

#ifdef CONFIG_POSIX_CPU_TIMERS_TASK_WORK
    struct posix_cputimers_work posix_cputimers_work;
#endif

    /* Process credentials: */

    /* Tracer's credentials at attach: */
    const struct cred __rcu     *ptracer_cred;

    /* Objective and real subjective task credentials (COW): */
    const struct cred __rcu     *real_cred;

    /* Effective (overridable) subjective task credentials (COW): */
    const struct cred __rcu     *cred;
```

`min_flt` and `maj_flt` count faults on the task. Day 2's copy-on-write fault is one of the events that increments them. The credential pointers are separate objects. The comment marks them copy-on-write: a task that changes credentials installs a new `struct cred` and drops its reference on the old one, which matters when another task still holds the previous object. A syscall authorizes against this object. ToyOS had no such object; the single user process was allowed to issue the teaching syscalls by being the process that `iretq` had entered. Linux records authority as data so that `exec` can change it (Day 3 calls `commit_creds`) and so that two tasks can carry different authority in the same kernel.

Files, namespaces, and signals, same struct:

```
    /* Filesystem information: */
    struct fs_struct        *fs;

    /* Open file information: */
    struct files_struct     *files;

#ifdef CONFIG_IO_URING
    struct io_uring_task        *io_uring;
#endif

    /* Namespaces: */
    struct nsproxy          *nsproxy;

    /* Signal handlers: */
    struct signal_struct        *signal;
    struct sighand_struct __rcu     *sighand;
    sigset_t            blocked;
    sigset_t            real_blocked;
    /* Restored if set_restore_sigmask() was used: */
    sigset_t            saved_sigmask;
    struct sigpending       pending;
```

`fs` is filesystem context: root, current directory, umask. `files` is the open-file table, indexed by descriptor number. `nsproxy` holds the namespaces this task lives in, including the PID namespace that interprets `pid` and `tgid`. `signal` is the process-wide signal state a thread group shares. `sighand` is the disposition table. `blocked` and `pending` belong to this task. The split is the reason a later lesson can say "the mask is per task" and "the disposition is per thread group" by pointing at two different fields.

The architecture-specific tail must stay last. Same struct, same tree:

```
    /*
     * New fields for task_struct should be added above here, so that
     * they are included in the randomized portion of task_struct.
     */
    randomized_struct_fields_end

    /* CPU-specific state of this task: */
    struct thread_struct        thread;

    /*
     * WARNING: on x86, 'thread_struct' contains a variable-sized
     * structure.  It *MUST* be at the end of 'task_struct'.
     *
     * Do not put anything below here!
     */
};
```

`thread` holds CPU-specific task state. On x86 the comment requires it at the end because `thread_struct` varies in size. User general-purpose registers at a syscall or interrupt boundary live in a `pt_regs` frame on the kernel stack. While the task is running in user mode, those values are in the core. When the scheduler switches away, `RIP` from Week 2 is stored with the task. The ELF file stores `e_entry`. The live instruction pointer is a property of the task.

---

5. Page tables live in `mm`, registers live on the task

In `include/linux/mm_types.h`, `struct mm_struct`, same tree, the root sits with the virtual-memory areas:

```
struct mm_struct {
    struct {
        struct vm_area_struct *mmap;        /* list of VMAs */
        struct rb_root mm_rb;
        u64 vmacache_seqnum;                   /* per-thread vmacache */
#ifdef CONFIG_MMU
        unsigned long (*get_unmapped_area) (struct file *filp,
                unsigned long addr, unsigned long len,
                unsigned long pgoff, unsigned long flags);
#endif
        unsigned long mmap_base;    /* base of mmap area */
        unsigned long mmap_legacy_base; /* base of mmap area in bottom-up allocations */
#ifdef CONFIG_HAVE_ARCH_COMPAT_MMAP_BASES
        /* Base adresses for compatible mmap() */
        unsigned long mmap_compat_base;
        unsigned long mmap_compat_legacy_base;
#endif
        unsigned long task_size;    /* size of task vm space */
        unsigned long highest_vm_end;   /* highest vma end address */
        pgd_t * pgd;
```

`mmap` is the list of virtual-memory areas. `pgd` is the page-directory root, a `pgd_t *`. Week 5's walk starts there: each index selects the next table, and the leaf names a frame and permission bits. ToyOS stores the same idea in `process->root` and loads it when `address_space_activate` runs. Linux loads `mm->pgd` when it switches to a task whose `active_mm` requires a different root.

A C pointer, from Week 2 and Week 3, is a virtual address interpreted by this root. The abstract machine's objects — the stack slot, the heap block, the string literal in `.rodata` — occupy mappings in this `mm`. The `task_struct`, the `struct cred`, and the descriptor table occupy kernel memory. User mode fetches and stores are translated through `mm->pgd` and fault if the leaf is absent or the permission bit rejects the access.

That is why a syscall argument that "is a pointer" still has to be copied. Week 8's `copy_from_user` refuses a kernel-range address and walks the user root one byte at a time. Linux draws the same boundary with a larger copy routine: the register contains a virtual address in the caller's `mm`, and the kernel copies bytes out under that translation before it trusts them. The page tables enforce the boundary on every user load. The copy enforces it at the moment the kernel chooses to read.

```
while this task is current
──────────────────────────
core registers  ↔  pt_regs on this task's kernel stack, and thread
page-table root ↔  active_mm->pgd
user virtual address 0x7fff…  walks that root
kernel object (files, cred, task)  lives outside that user map
```

Two ordinary processes therefore have two roots. Day 2 will show that the roots can still point at the same physical frame until one of them writes.

---

6. `getpid` returns the thread-group id in a namespace

In `kernel/sys.c`, `SYSCALL_DEFINE0(getpid)` and `SYSCALL_DEFINE0(gettid)`, same tree:

```
/**
 * sys_getpid - return the thread group id of the current process
 *
 * Note, despite the name, this returns the tgid not the pid.  The tgid and
 * the pid are identical unless CLONE_THREAD was specified on clone() in
 * which case the tgid is the same in all threads of the same group.
 *
 * This is SMP safe as current->tgid does not change.
 */
SYSCALL_DEFINE0(getpid)
{
    return task_tgid_vnr(current);
}

/* Thread ID - the internal kernel "pid" */
SYSCALL_DEFINE0(gettid)
{
    return task_pid_vnr(current);
}
```

`task_tgid_vnr` in `include/linux/sched.h` calls `__task_pid_nr_ns(tsk, PIDTYPE_TGID, NULL)`. The `NULL` namespace argument means the caller's current PID namespace. The number is a view of a `struct pid` as seen from the caller's namespace. `gettid` uses `PIDTYPE_PID` and returns the per-task id. The comment states the rule this tree implements: `tgid` and `pid` match unless the task was created with `CLONE_THREAD`, in which case every task in the group shares one `tgid` and each has its own `pid`.

ToyOS `SYS_GETPID` writes `init_process.pid` into the saved `RAX`. There is one namespace, the number 1, and one task. Linux inserts a namespace lookup between the task and the integer a program sees. A container and its host can report different integers for one `struct pid`. After `usage` drops and the pid is freed, the namespace may assign that integer to a later task. A lone integer in a log is evidence together with the namespace and the time of the observation.

`/proc/self` is that lookup from the reading task. The lab opens it, and opens `/proc/<getpid()>/status`, and both paths name the same task for a single-threaded process because `getpid` returned `tgid` and `/proc/self` resolves the reader.

---

7. Descriptors, credentials, and lifetime are referenced objects

A descriptor number is an index in the `fdtable` hung off `files_struct`. In `include/linux/fdtable.h` the table has a reference count, a pointer to the current `fdtable`, a `close_on_exec` bitmap, and an array of `struct file *`. In `include/linux/fs.h`, `struct file` carries `f_count`, `f_pos`, `f_flags`, `f_mode`, and `f_path`. Three different facts travel under the word "file":

```
int fd                          index in this task's fdtable
fdtable.fd[fd]                  struct file *   (the open-file description)
struct file.f_pos               shared offset, with f_count references
struct file.f_path              which inode, pipe, or socket
fdtable.close_on_exec bit       belongs to this index, in this table
```

Day 2's ordinary fork copies the table and takes a new reference on each `struct file`, so the offset is shared and the index is private. Day 3's `FD_CLOEXEC` closes one index at image replacement because the bit lives in the table, beside the index. Closing an index drops one `f_count`. The description lives as long as some index, some kernel path, or some other task still holds it. Week 9's ToyOS file table is one array inside one kernel; Linux makes the array an object tasks can share or duplicate.

Credentials work the same way. `real_cred` and `cred` are pointers. Changing uids replaces the pointer. The previous object stays until its last user releases it. `/proc/PID/status` prints a projection of the ids. The authorization check reads the object.

`exit_state` and `usage` are the lifetime pair on the task itself. The task can be finished as an execution stream and still occupy a pid while `usage` keeps the object alive for the parent. That is the structural reason a process is a kernel object with a life cycle, and an ELF file is an input that many such objects can name.

---

8. Execution trace: `proc_lab` reads its own task

The program in projects/linux-system-programming/src/proc_lab.c calls `getpid`, opens `/proc/<that>/status`, prints the `Pid:` and `PPid:` lines, prints the first line of `/proc/self/stat`, and `readlink`s `/proc/self/exe`. The following trace is one run of that program after its image is already installed. Numeric addresses and ids below are a single illustrated run. The wrapper prints the real ones.

```
shell task                          proc_lab task
RIP in the shell's text             RIP in proc_lab / libc text
mm_shell->pgd installed             mm_lab->pgd installed when it runs
files: shell's descriptors          files: lab's descriptors
pid/tgid: shell's ids               pid == tgid: this process id
real_parent: shell's parent         real_parent: the shell (or its child
                                    if the shell forked first)
```

1. The scheduler makes the `proc_lab` task `current`. The core's page-table root register receives `active_mm->pgd`, which is `mm->pgd` for this user task. User `RIP` is restored from the task's saved frame to an instruction in the mapped image. That image was built from the ELF `PT_LOAD` segments. The file's `e_entry` was an input to that mapping. The live `RIP` is a register.
2. `main` calls `getpid`. Libc places the syscall number in the ABI register and crosses the privilege boundary. On this x86-64 host the usual instruction is `syscall`. ToyOS used `int $0x80` and a DPL-3 interrupt gate; both paths arrive in the kernel with the user registers saved on the task's kernel stack and with `current` equal to this `task_struct`. The handler is `sys_getpid`. It returns `task_tgid_vnr(current)`. For this single-threaded task, `pid` and `tgid` hold the same number, so the integer matches the per-task id as well. The return path writes that integer into the user return register and resumes user `RIP` at the instruction after the syscall. ToyOS did the same write to `frame->rax` for `SYS_GETPID`.
3. `snprintf` builds `/proc/<tgid>/status`. `fopen` allocates a descriptor index in this task's `files`, pointing at a new `struct file` whose `f_path` is that proc inode. The reads that follow run in kernel mode, generating text from this task: the `Pid:` line from the thread-group id, the `PPid:` line from the parent. Those strings are copied back to the user buffer. The user buffer's address is a virtual address in `mm_lab`. The copy is the Linux form of Week 8's checked user copy.
4. `fopen("/proc/self/stat")` resolves `self` as the reading task, the same `current`. The first line includes the `comm` field stored on the task and the task state. `readlink("/proc/self/exe")` returns the path of the file installed as this `mm`'s executable. That file is the ELF program. The task holding `mm` is the process. The link can name the same inode for every run. The task, the `mm`, and the `pgd` are new each run.
5. A second start of the same path repeats the construction: another `task_struct`, another `mm`, another `pgd`, another `files`, another pair of credential references, another `pid` lookup in the namespace. Both tasks can be runnable. Each has its own saved `RIP`. A store by one walks only the root installed while it is current. The ELF bytes on disk stay one file. If the mappings are private, Day 2's copy-on-write decides whether a write allocates a new frame. The two processes already exist before that question comes up, because they already have two tasks.

The wrapper `challenges/day-001-process-model.sh` builds this program, prints ELF class, type, machine, and entry, and runs it. The stable line is `proc_inspection=ok`. The PID, PPID, and executable path are observations of one task on one host.

---

9. Architecture lens — x86-64, AArch64, and RISC-V

The graph in `task_struct` is C. `mm`, `files`, `fs`, `real_cred`, `signal`, `pid`, and `tgid` are the same members on every architecture this tree builds. What the ISA changes is where a resume register and a page-table root live in the core, and which instruction enters the kernel.

```
               user resume          page-table root        syscall entry
x86-64         RIP, in pt_regs.ip   CR3 ← mm->pgd          syscall
AArch64        PC                   TTBR0_EL1 ← mm->pgd    svc
RISC-V         PC                   satp ← mm->pgd         ecall
```

On x86-64, `thread_struct` is the variable-sized tail the comment in `sched.h` warns about, and the user general-purpose registers at a privilege crossing sit in `pt_regs` on the kernel stack. `iretq` in ToyOS is the teaching form of "restore RIP, CS, RFLAGS, RSP, SS from a frame." Linux's `syscall` return path restores the same user-visible contract through a different entry instruction. AArch64 and RISC-V store their register frames in their own `pt_regs` layouts and install their own page-table root register from the same `mm->pgd` pointer. The fault and syscall entry stubs differ. Once `current` is set, `sys_getpid` runs the same C: it reads `tgid` in the caller's PID namespace.

The ELF `e_machine` field selects which of those machines the file is for. `readelf -h` on the lab binary prints that machine. It prints an entry address in the file's image. The live `RIP` is a register of the running task.

---

10. Mastery: Explain · Draw · Observe · Build

The lab is evidence after this model. From the week directory, run `challenges/day-001-process-model.sh`. It builds `projects/linux-system-programming` and runs `proc_lab` on itself.

Explain

- A program is an ELF file: program headers, `e_entry`, and bytes. A process is a `task_struct` plus `mm`, `files`, `fs`, `real_cred` / `cred`, `signal`, and a pid in a namespace.
- `getpid` returns `task_tgid_vnr(current)`. For a single-threaded task, `pid` and `tgid` hold the same number. `gettid` returns the per-task id.
- Registers live with the task (`thread`, `pt_regs` on `stack`). Page tables live in `mm->pgd`. Week 2's `RIP` and Week 5's walk meet in those two places.
- ToyOS `struct process` is the small version: one root, one entry, one kernel stack, one PID, entered by `iretq`. Linux adds sharing, credential objects, namespaces, and a task lifetime separate from the ELF file.
- Starting one ELF twice allocates two tasks. The inode of the executable can be shared. Each run has its own page-table root and its own register save area.

Draw

1. The `task_struct` graph from section 3, with an arrow from `mm` to `pgd` and an arrow from `files` to `struct file`.
2. One core, with `RIP` and the page-table root register, and the two kernel objects those registers are loaded from.
3. Two runs of `proc_lab`: two tasks, two `mm`s, one ELF inode.

Observe

The wrapper prints `uname`, the ELF header (class, type, machine, entry), `Pid:` and `PPid:` from status, the `stat` line, `exe=`, and `proc_inspection=ok`. Record which of those come from the file (ELF header, hash) and which come from the live task (pid, ppid, exe link, stat). Run it twice and mark the fields that changed. The script's evidence is the calling process: status, stat, exe, and the ELF header.

Build

In the week-10 project, extend the observation only through its existing interfaces: predict `Pid` versus `PPid` before the run, then point at the `task_struct` fields that supply each printed line. Keep the experiment on the calling process. The wrapper's LIMIT line is the boundary of the evidence: the calling process, its own pid, and the fields `proc_lab` prints.

Why? notebook

1. Why can the ELF file stay unmodified while two processes run it?
2. Why does `mm` exist separately from `task_struct`?
3. Why does `getpid` read `tgid`?
4. Why is a PID an integer only after a namespace lookup?
5. Why does the kernel stack belong to the task, next to the user register frame?
6. Why is `close_on_exec` a bit in the descriptor table, beside `f_pos` living in `struct file`?
7. Why can `exit_state` be meaningful after user mappings are gone?
8. Why did ToyOS need `copy_from_user` once the page-table root became per process?

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Process Model

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-001-process-model.sh
```

- View lab source on GitHub

Day 2 — Fork, Copy-on-Write, and Lineage

Target time: approximately 3 hours

- Why a new task shares frames until a store: 30 minutes
- `fork` into `kernel_clone`, and what `copy_mm` / `copy_files` share: 45 minutes
- The write-protect and the fault that copies a frame: 50 minutes
- One trace of two stores to the same virtual address: 30 minutes
- Mastery, then the wrapper as evidence: 25 minutes

> Parent and child have just returned from `fork`. Both will store through the same virtual address. Why does the first store fault, and why does that fault allocate a frame for only one of them?

Day 1 separated the ELF file from the live task. Fork is how a task acquires a child task. The child needs its own registers, because Week 2's `RIP` and the general-purpose registers are per execution stream. The child needs its own page-table root, because Week 5's walk is per address space and Day 1 put that root in `mm->pgd`. Copying every physical frame at the `fork` call would make the cost proportional to the whole address space, and a child that immediately `exec`s (Day 3) would throw the copy away. The mechanism that resolves this is copy-on-write: duplicate the page tables, clear the writable permission on private writable mappings, and copy a frame on the store that first needs a private version.

ToyOS has a single `struct process`. Week 7 switches kernel tasks that share one set of page tables and copies only the saved registers. Linux fork adds a second task and a second `mm`, then uses the page-table permission bits Week 5 already taught so the second address space can still name the same frames.

The excerpts are from the local tree `5.10.134-15_rc1-10333-gc9b2385d6793` (commit `c9b2385d6793`), a 5.10-based tree. Later mainline code moves these functions. The lines below are this revision.

---

1. `fork` is a thin entry into `kernel_clone`

In `kernel/fork.c`, `SYSCALL_DEFINE0(fork)`, same tree. x86-64 and arm64 in this tree define `__ARCH_WANT_SYS_FORK`, so the syscall exists there:

```
#ifdef __ARCH_WANT_SYS_FORK
SYSCALL_DEFINE0(fork)
{
#ifdef CONFIG_MMU
    struct kernel_clone_args args = {
        .exit_signal = SIGCHLD,
    };

    return kernel_clone(&args);
#else
    /* can not support in nommu mode */
    return -EINVAL;
#endif
}
#endif
```

The arguments are only an exit signal. Clone flags stay zero: this path asks for a new `mm`, a new file table, a new signal disposition table, and a new filesystem context. `SIGCHLD` is the signal the child will deliver to its parent when it exits. That is the lineage contract, stored later on the child as `exit_signal`.

`kernel_clone` is the common body. In `kernel/fork.c`, `kernel_clone`, same tree, after it has chosen whether a ptracer wants an event:

```
    p = copy_process(NULL, trace, NUMA_NO_NODE, args);
    add_latent_entropy();

    if (IS_ERR(p))
        return PTR_ERR(p);

    /*
     * Do this prior waking up the new thread - the thread pointer
     * might get invalid after that point, if the thread exits quickly.
     */
    trace_sched_process_fork(current, p);

    pid = get_task_pid(p, PIDTYPE_PID);
    nr = pid_vnr(pid);

    if (clone_flags & CLONE_PARENT_SETTID)
        put_user(nr, args->parent_tid);

    if (clone_flags & CLONE_VFORK) {
        p->vfork_done = &vfork;
        init_completion(&vfork);
        get_task_struct(p);
    }

    if (IS_ENABLED(CONFIG_LRU_GEN) && !(clone_flags & CLONE_VM)) {
        /* lock the task to synchronize with memcg migration */
        task_lock(p);
        lru_gen_add_mm(p->mm);
        task_unlock(p);
    }

    wake_up_new_task(p);
```

`copy_process` allocates the child `task_struct` and fills it. On success, `kernel_clone` reads the child's pid in the caller's namespace (`pid_vnr`) and wakes the child with `wake_up_new_task`. The function then returns `nr` to the parent. The parent's user return value is that pid. The child's user return value is installed earlier, inside `copy_thread`, as part of building a resume frame that will run `ret_from_fork` and then the user instruction after the syscall.

`copy_process` in this file calls the copiers in a fixed order. The ones that define today's sharing are `copy_files`, `copy_fs`, `copy_sighand`, `copy_signal`, `copy_mm`, then `copy_thread`, then `alloc_pid` in the child's PID namespace. Each copier either takes a reference on the parent's object or allocates a new one. Failure unwinds the ones that already succeeded. The child task is one object. The address space, the file table, and the pid are other objects, referenced from it, which is the Day 1 graph being duplicated under flags.

On x86-64, `copy_thread` in `arch/x86/kernel/process.c` builds that resume frame. For an ordinary user task it copies the parent's `pt_regs` and then sets the child's syscall return register to zero:

```
    /* Kernel thread ? */
    if (unlikely(p->flags & PF_KTHREAD)) {
        p->thread.pkru = pkru_get_init_value();
        memset(childregs, 0, sizeof(struct pt_regs));
        kthread_frame_init(frame, sp, arg);
        return 0;
    }

    /*
     * Clone current's PKRU value from hardware. tsk->thread.pkru
     * is only valid when scheduled out.
     */
    p->thread.pkru = read_pkru();

    frame->bx = 0;
    *childregs = *current_pt_regs();
    childregs->ax = 0;
    if (sp)
        childregs->sp = sp;
```

Earlier in the same function, `frame->ret_addr` is `ret_from_fork` and `p->thread.sp` points at that frame. When the child is scheduled, the kernel resume path pops into `ret_from_fork`, which returns to user mode with `RAX` already 0. The parent's `RAX` is whatever `kernel_clone` returned, the child's pid. Both tasks continue at the user instruction after the syscall. They have different register files because they have different `pt_regs`. They have the same virtual address for that instruction because the page tables were duplicated from the parent.

`vfork` in the same file passes `CLONE_VFORK | CLONE_VM` and the same `SIGCHLD`. `CLONE_VM` is the flag that shares the `mm`. The plain `fork` syscall leaves it clear.

---

2. `copy_mm` shares the address space only under `CLONE_VM`

In `kernel/fork.c`, `copy_mm`, same tree. The function first clears the child's fault and switch counters, then decides:

```
    tsk->mm = NULL;
    tsk->active_mm = NULL;

    /*
     * Are we cloning a kernel thread?
     *
     * We need to steal a active VM for that..
     */
    oldmm = current->mm;
    if (!oldmm)
        return 0;

    /* initialize the new vmacache entries */
    vmacache_flush(tsk);

    if (clone_flags & CLONE_VM) {
        mmget(oldmm);
        mm = oldmm;
        goto good_mm;
    }

    retval = -ENOMEM;
    mm = dup_mm(tsk, current->mm);
    if (!mm)
        goto fail_nomem;

good_mm:
    tsk->mm = mm;
    tsk->active_mm = mm;
    return 0;
```

A kernel thread has `current->mm == NULL`. The child then also has a null `mm` and will borrow an `active_mm` when it runs, which is the Day 1 kernel thread case and the Week 7 shared-page-table case.

`CLONE_VM` takes another reference (`mmget`) and stores the parent's `mm` into the child. One `mm` means one `pgd`, one list of VMAs, one set of page tables. A store by either task walks the same tables. Threads that share an address space pass this flag. `vfork` passes it so the child can run in the parent's address space until it `exec`s or exits; the parent waits in `kernel_clone` on `vfork_done`.

Plain `fork` fails the flag test and calls `dup_mm`. `dup_mm` allocates a new `mm_struct`, copies the VMA layout, and duplicates the page tables through `dup_mmap`. The child receives `tsk->mm` and `tsk->active_mm` pointing at that new descriptor. Two roots. Week 5's walk in the child starts at the child's `pgd` once the scheduler installs it.

---

3. Duplicating a page table write-protects a COW mapping

A shared mapping (`VM_SHARED`) is supposed to make one task's store visible to the other. A private mapping keeps the store inside the storing address space. Copy-on-write still starts from one frame: it removes write permission from both PTEs, the hardware faults the first store, and the fault handler allocates a private frame.

`is_cow_mapping` in `include/linux/mm.h`, same tree, is the predicate. `page_needs_cow_for_dma` uses it when a pin means the frame must be copied during `fork` itself:

```
static inline bool is_cow_mapping(vm_flags_t flags)
{
    return (flags & (VM_SHARED | VM_MAYWRITE)) == VM_MAYWRITE;
}

/*
 * This should most likely only be called during fork() to see whether we
 * should break the cow immediately for a page on the src mm.
 */
static inline bool page_needs_cow_for_dma(struct vm_area_struct *vma,
                      struct page *page)
{
    if (!is_cow_mapping(vma->vm_flags))
        return false;
```

A mapping is copy-on-write when it may be written and it is private. `VM_MAYWRITE` is the permission the VMA is allowed to have, which stays set even after the PTE write bit is cleared. The fault handler uses it later to decide that the faulting task is allowed to become writable again after the copy.

In `mm/memory.c`, `copy_present_pte`, same tree, after the page's map count has been duplicated:

```
    /*
     * If it's a COW mapping, write protect it both
     * in the parent and the child
     */
    if (is_cow_mapping(vm_flags) && pte_write(pte)) {
        ptep_set_wrprotect(src_mm, addr, src_pte);
        pte = pte_wrprotect(pte);
    }

    /*
     * If it's a shared mapping, mark it clean in
     * the child
     */
    if (vm_flags & VM_SHARED)
        pte = pte_mkclean(pte);
    pte = pte_mkold(pte);

    if (!userfaultfd_wp(dst_vma))
        pte = pte_clear_uffd_wp(pte);

    set_pte_at(dst_vma->vm_mm, addr, dst_pte, pte);
    return 0;
```

`ptep_set_wrprotect` clears the writable bit in the parent's live PTE. `pte_wrprotect` clears it in the value that `set_pte_at` installs in the child. Both PTEs still contain the same page-frame number. Above this block, when the leaf names an ordinary page, the function has already called `get_page` and `page_dup_rmap`, so the frame's reference count and map count include the child. The child's page-table pages themselves are new. The data frame is shared.

A page that is pinned for DMA is handled just before this path, in `copy_present_page`. If `page_needs_cow_for_dma` says the parent has pinned this anonymous page, the kernel copies the frame during `fork` and installs the copy in the child immediately. The pin must keep seeing the parent's frame. The common case, an ordinary anonymous or private file page with no such pin, takes the write-protect path.

Read-only PTEs stay read-only. There is nothing to unshare until something makes the mapping writable. Executable text that is mapped read-only can remain one frame for parent and child for the rest of their lives. The instruction both tasks return to can be that shared frame, fetched through two roots.

```
before fork                         after fork, before any store
parent PTE                          parent PTE              child PTE
V → PFN 0x12000, writable           V → PFN 0x12000, RO     V → PFN 0x12000, RO
one mapcount                        mapcount includes both tasks
                                    two pgd roots
```

---

4. The first store faults and copies; the last reference may reuse

A store to a present PTE whose writable bit is clear enters the fault handler with the address, the VMA, and the original PTE. In `mm/memory.c`, `do_wp_page`, same tree, the anonymous decision is:

```
    if (PageAnon(vmf->page)) {
        struct page *page = vmf->page;

        /* PageKsm() doesn't necessarily raise the page refcount */
        if (PageKsm(page) || page_count(page) != 1)
            goto copy;
        if (!trylock_page(page))
            goto copy;
        if (PageKsm(page) || page_mapcount(page) != 1 || page_count(page) != 1) {
            unlock_page(page);
            goto copy;
        }
        /*
         * Ok, we've got the only map reference, and the only
         * page count reference, and the page is locked,
         * it's dark out, and we're wearing sunglasses. Hit it.
         */
        unlock_page(page);
        wp_page_reuse(vmf);
        return VM_FAULT_WRITE;
    } else if (unlikely((vma->vm_flags & (VM_WRITE|VM_SHARED)) ==
                    (VM_WRITE|VM_SHARED))) {
        return wp_page_shared(vmf);
    }
copy:
    /*
     * Ok, we need to copy. Oh, well..
     */
    get_page(vmf->page);

    pte_unmap_unlock(vmf->pte, vmf->ptl);
    return wp_page_copy(vmf);
```

Right after `fork`, the frame is mapped by parent and child, so `page_count` and `page_mapcount` are above one. The faulting task falls through to `wp_page_copy`. `wp_page_reuse` is the other exit: the faulting task holds the only map and the only page reference, so the handler marks the existing PTE writable and dirty. That happens for the second task once the first task has dropped its map of this frame, and it also happens for a page that was already private. The write bit is still clear until this fault, because `copy_present_pte` cleared it. Reuse avoids a second allocation. It still is a write fault.

The copy itself is `wp_page_copy`. In `mm/memory.c`, same tree, the allocation and the byte copy:

```
static vm_fault_t wp_page_copy(struct vm_fault *vmf)
{
    struct vm_area_struct *vma = vmf->vma;
    struct mm_struct *mm = vma->vm_mm;
    struct page *old_page = vmf->page;
    struct page *new_page = NULL;
    pte_t entry;
    int page_copied = 0;
    struct mmu_notifier_range range;
    int ret;

    if (unlikely(anon_vma_prepare(vma)))
        goto oom;

    if (is_zero_pfn(pte_pfn(vmf->orig_pte))) {
        new_page = alloc_zeroed_user_highpage_movable(vma,
                                  vmf->address);
        if (!new_page)
            goto oom;
    } else {
        new_page = alloc_page_vma(GFP_HIGHUSER_MOVABLE, vma,
                vmf->address);
        if (!new_page)
            goto oom;

        ret = cow_user_page(new_page, old_page, vmf);
```

A zero page becomes a fresh zeroed frame. Any other page is allocated and filled by `cow_user_page`, which copies the source frame's bytes. The function then charges the new page, locks the PTE again, and checks that the PTE still matches `orig_pte` (another fault might have won the race). On a match it builds a PTE from `mk_pte(new_page, vma->vm_page_prot)`, marks it young, and `maybe_mkwrite(pte_mkdirty(entry), vma)` sets the writable bit because the VMA has `VM_WRITE`. It clears the old PTE, flushes that translation, adds the new page as an anonymous rmap, and `set_pte_at_notify` installs the new PTE in the faulting `mm` only. Then `page_remove_rmap` drops the faulting task's map of the old page. The other task's PTE is untouched: same virtual address, old frame, write bit still clear.

The fault returns, the store instruction is restarted, and the writable PTE lets the store complete into the new frame. `min_flt` on that task counts the fault. The other task's registers and its PTE stay as they were.

---

5. Execution trace: one virtual address, two stores

Illustrated values, one anonymous private page, for a single-threaded parent. The lab prints pids and the pipe payload. This trace is the path the functions above take.

```
V = 0x7fff00001000          illustrative user virtual address
parent, before fork:
  mm_p->pgd walks V → PFN 0x12000, present, writable
  bytes at that frame: integer 1
```

1. The parent executes the `fork` syscall. `SYSCALL_DEFINE0(fork)` calls `kernel_clone` with `exit_signal = SIGCHLD` and empty flags. `copy_process` duplicates the task. `copy_mm` calls `dup_mm`. `copy_present_pte` write-protects V in both page tables. Both roots now translate V to PFN `0x12000` with the writable bit clear. Map count is two.
2. `copy_thread` on x86-64 copies `pt_regs` and sets `childregs->ax = 0`. `alloc_pid` assigns a pid in the namespace. `wake_up_new_task` queues the child. `kernel_clone` returns `pid_vnr` to the parent. The parent's libc wrapper returns that positive pid. The child resumes at `ret_from_fork` and returns to user mode with `RAX == 0`. Both user `RIP` values name the instruction after the syscall. That instruction page can still be the shared read-only frame. The data page at V is shared and write-protected.
3. The parent stores `2` through V. The parent's walk uses `mm_p->pgd`. The leaf is present and the writable bit is clear, so the core raises a page fault. `do_wp_page` sees an anonymous page with `page_count != 1` and calls `wp_page_copy`. The handler allocates PFN `0x13000`, `cow_user_page` copies the integer `1` into it, and the parent's PTE for V becomes present, writable, and pointing at `0x13000`. `page_remove_rmap` drops the parent's map of `0x12000`. The restarted store writes `2` into `0x13000`.
4. The child's PTE for V still names `0x12000` and is still write-protected. A load by the child still returns `1`. The virtual address is the same number. The page-table root is `mm_c->pgd`. Week 5's walk starts at the root installed for `current`, so one virtual address can name two frames.
5. The child stores `3` through V. The child's walk faults. `do_wp_page` runs on the child's `mm`. If the child is now the only mapper and the only reference, the handler calls `wp_page_reuse` and marks the existing PTE writable; the store writes `3` into PFN `0x12000`. If a reference remains — the count checks in `do_wp_page` are the authority — the handler calls `wp_page_copy` again and the child gets a third frame. Either way the parent's frame `0x13000` still holds `2`. The two stores used one virtual address and two `mm` objects.

```
after both stores (reuse on the second fault)
parent:  V → PFN 0x13000 writable, bytes 2
child:   V → PFN 0x12000 writable, bytes 3
```

A shared mapping (`VM_SHARED` and `VM_WRITE`) takes `wp_page_shared` in `do_wp_page` and marks the existing page writable for both tasks. Private anonymous memory, including the C stack and a `MAP_PRIVATE` anonymous `mmap`, takes the copy path. That is why a global in `.data` and a local on the stack diverge after `fork` on the first store, while a `MAP_SHARED` region stays one frame.

---

6. The file table is private; the open-file description is shared

Memory and descriptors answer different sharing questions. In `kernel/fork.c`, `copy_files`, same tree:

```
static int copy_files(unsigned long clone_flags, struct task_struct *tsk)
{
    struct files_struct *oldf, *newf;
    int error = 0;

    /*
     * A background process may not have any files ...
     */
    oldf = current->files;
    if (!oldf)
        goto out;

    if (clone_flags & CLONE_FILES) {
        atomic_inc(&oldf->count);
        goto out;
    }

    newf = dup_fd(oldf, NR_OPEN_MAX, &error);
    if (!newf)
        goto out;

    tsk->files = newf;
    error = 0;
out:
    return error;
}
```

`CLONE_FILES` increments `files_struct.count` and leaves `tsk->files` pointing at the parent's table. Both tasks then share descriptor indices: a `close` in one is a `close` in the other. Threads that share the file table pass this flag. Plain `fork` leaves it clear, so `dup_fd` allocates a new `files_struct` with `count` 1.

`dup_fd` in `fs/file.c` copies the open-fd and `close_on_exec` bitmaps, then walks the parent's `struct file *` array. For each non-null pointer it calls `get_file`, which increments `struct file.f_count`, and stores that same pointer in the child's table. The child has a private index space and a shared open-file description. `f_pos` lives in `struct file`, so a `read` that advances the offset is visible through the other task's copy of the index. `close` on one index drops one `f_count` and clears that task's slot. The other task's slot still holds the pointer.

```
after fork, without CLONE_FILES

parent files_struct                child files_struct
fd 3 ──────────────┐         ┌──── fd 3
                   ▼         ▼
              struct file  (one f_pos, f_count ≥ 2)
                   │
                pipe / inode
```

The week-10 tree program uses this directly. In [projects/linux-system-programming/src/process_lab.c](), `mode_tree` creates a pipe, forks twice, and each child writes three `long`s: its loop index, `getpid()`, and `getppid()`. The write end is a `struct file` duplicated by `dup_fd`. Each child `close`s its copy of the read end and, after the write, its copy of the write end. The parent's read end remains valid because its `fdtable` slot is a separate reference. The two children have different pids and the same parent pid, because each `copy_process` set

`real_parent` to the task that called `fork`, and `getppid` reads that relationship in the caller's namespace. Record order on the pipe follows whichever child the scheduler runs. The pid values are the lineage. The shared pipe buffer is the file-description rule.

`copy_fs` is the same pattern for cwd and root: `CLONE_FS` bumps `fs_struct.users`, and plain `fork` calls `copy_fs_struct`. A later `chdir` in the child updates the child's `fs`, and the parent's cwd stays.

---

7. The child is a new task in the parent's lineage

`exit_signal = SIGCHLD` from the `fork` syscall is stored on the child. When the child exits, the parent is the `SIGCHLD` recipient Day 1 named as `parent` (and, absent a tracer, `real_parent`). The child's `task_struct` is linked on the parent's `children` list. `alloc_pid` creates the `struct pid` in `nsproxy->pid_ns_for_children`. `getpid` in the child returns the new `tgid`. `getpid` in the parent returns the parent's `tgid`, unchanged, because `fork` created a different task.

Two children of one parent, as in `mode_tree`, are two `task_struct`s, two `mm`s, two pids, and one parent pointer. Their address spaces start as write-protected duplicates. Their pipe ends start as extra references on the same `struct file`. Those are independent axes. Equal descriptor integers across the two children mean "this index was copied," and the sharing of `f_pos` comes from `get_file` on the same pointer.

Day 4 will wait on these children and read `exit_code`. Today the creation side is enough: the parent received the pid from `kernel_clone`, the child received 0 from `copy_thread`, and both are runnable tasks with a lineage edge between them.

---

8. Architecture lens — x86-64, AArch64, and RISC-V

Copy-on-write is a page-table permission fact plus a fault handler in generic `mm/`. `copy_mm`, `is_cow_mapping`, `do_wp_page`, and `wp_page_copy` are that C. The ISA supplies the PTE encoding inside `pte_wrprotect` / `ptep_set_wrprotect`, and it supplies the store fault that enters the handler.

```
               legacy fork syscall in this tree     store fault into do_wp_page
x86-64         yes (__ARCH_WANT_SYS_FORK)           page fault, present+read-only
AArch64        yes (__ARCH_WANT_SYS_FORK)           data abort, permission
RISC-V         no; clone / clone3 are compiled      page fault, store vs PTE
```

`arch/riscv/include/asm/unistd.h` in this tree defines `__ARCH_WANT_SYS_CLONE`. The legacy `fork` syscall is compiled for the architectures that define `__ARCH_WANT_SYS_FORK`, including x86 and arm64 in this tree. User code on RISC-V reaches `kernel_clone` through `SYSCALL_DEFINE5(clone)` or `clone3`. The C library on x86-64 often does the same: `fork()` in libc is a `clone` system call with flags that still mean "new mm, new files, `SIGCHLD`." `strace` on the lab can print `clone` for a C `fork()`. Both enter `kernel_clone`. The flag word is what `copy_mm` and `copy_files` test.

The child's zero return is written by architecture code. The x86-64 `copy_thread` quoted above stores it in `childregs->ax` and resumes through `ret_from_fork`. AArch64 and RISC-V do the analogous store in their own `copy_thread`, into the ABI return register of that ISA. The generic function returns the pid only to the task that called `kernel_clone`, which is the parent.

The fault entry differs, and the PTE bit that means "writable" differs. After entry, the handler is `do_wp_page` on the faulting `mm`, and it allocates a frame only for that `mm`.

---

9. Mastery: Explain · Draw · Observe · Build

The mechanism above is the lesson. The wrapper is the evidence you can check on this host. From the week directory, run `challenges/day-002-fork-cow.sh`. It builds the canonical project and runs `process_lab tree`.

Explain

- `SYSCALL_DEFINE0(fork)` calls `kernel_clone` with `exit_signal = SIGCHLD` and no clone flags. `copy_process` builds the child. The parent receives `pid_vnr`. On x86-64 the child receives `childregs->ax = 0`.
- `copy_mm` shares `mm` when `CLONE_VM` is set and calls `dup_mm` otherwise. Plain `fork` duplicates the address space.
- `copy_present_pte` write-protects both PTEs of a COW mapping (`is_cow_mapping`: private and `VM_MAYWRITE`) and keeps the same frame. The first store faults into `do_wp_page` and `wp_page_copy`, which installs a new frame in the faulting `mm` only.
- `copy_files` shares `files_struct` under `CLONE_FILES`. Plain `fork` calls `dup_fd`, which `get_file`s each `struct file`. The index is private. `f_pos` is shared.
- Two children of `mode_tree` have one parent and a shared pipe description. Their stacks are separate address spaces under copy-on-write.

Draw

1. Parent and child page tables for virtual address V, before either store, both naming one PFN with the writable bit clear.
2. The same diagram after the parent's store: parent PTE names a new PFN, child PTE still names the old one.
3. Two `files_struct`s and one `struct file` for the pipe write end, with `f_count` at least 2.

Observe

The wrapper prints one line per child: `node`, `pid`, `ppid`, and the root pid. Predict two distinct child pids, one shared ppid equal to the root, and an unspecified order. If `strace` exists, the wrapper keeps `clone`, `fork`, `wait`, and `exit` lines. A `clone` line is the libc entry into `kernel_clone`. The wrapper's LIMIT line records the boundary of the run: lineage and the pipe. Page-fault counts and `/proc/pid/pagemap` stay in the source trace of section 5. The pipe lines are the runtime evidence for lineage and for a shared open-file description.

Build

Using the existing `process_lab` tree mode, write down for each `close` in the child which reference drops (`f_count` on which end) and which of the parent's descriptors must stay open for `checked_read` to finish. Then name the `copy_files` / `dup_fd` path that made those descriptors exist. Physical frames stay a source explanation unless you add a fault counter you can account for. The wrapper measures lineage and the shared pipe.

Why? notebook

1. Why is the page table duplicated at `fork` while the frame stays shared?
2. Why does `VM_MAYWRITE` stay set after `pte_wrprotect`?
3. Why can the second writer's fault call `wp_page_reuse`?
4. Why does `fork` pass `SIGCHLD` and leave `CLONE_VM` clear?
5. Why does `vfork` set `CLONE_VM`?
6. Why does `dup_fd` call `get_file` on the existing `struct file`?
7. Why can `strace` show `clone` for a C `fork()`?
8. Why do parent and child agree on the virtual address of a stack slot and disagree on the frame after one of them stores?

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Fork Cow

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-002-fork-cow.sh
```

- View lab source on GitHub

Day 3 — Exec Replaces the Image and Keeps the Task

Target time: approximately 3 hours

> A child has just called `execve("/bin/sh", ...)`. The call succeeds. Which `task_struct` is running, which `mm` is installed, and which instruction does `RIP` name?

Day 1 made a process a task plus referenced objects, and a program an ELF file. Day 2 showed how `fork` allocates a second task and a second `mm`, write-protecting private frames so the copy stays cheap. The usual reason that copy stays cheap is this call: the child exists only to become a different program. `execve` replaces the user image of the calling task. The pid remains the pid of that task, for the single-threaded case this week runs. The page tables, the user registers, the argument strings, and the dispositions of caught signals are rebuilt from the new ELF file.

The excerpts are from the local tree `5.10.134-15_rc1-10333-gc9b2385d6793` (commit `c9b2385d6793`). It is a 5.10-based tree. Names and line placement on current mainline differ.

---

1. Replacement is a property of the calling task

`fork` answers "who is the new task?" `execve` answers "what user image does this task run next?" Shells need both because a shell must remain the shell after starting a command. The sequence is the Day 2 child — its own `task_struct`, its own pid, a duplicated `mm` — followed by this call, which frees that duplicated `mm` and installs a new one. The parent's task is untouched. The child's task continues, with a new memory descriptor and a new user `RIP`.

ToyOS `process_init` loads one ELF and `process_start` enters it with `iretq`. Linux runs those same steps — check the header, map each `PT_LOAD`, build a user stack, enter ring 3 — against an existing task. `mmput` drops the old root, `activate_mm` installs the new one, and `pid` and `tgid` stay on that `task_struct` when the thread group has one member.

Before `bprm->point_of_no_return`, a failed `execve` returns a negative errno to the old image and libc reports `-1`. After `begin_new_exec` sets that flag, `exec_mmap` has installed the new `mm`, and a later failure ends the task with a fatal signal.

---

2. The system call builds a `linux_binprm` and hands it to the format

In `fs/exec.c`, `SYSCALL_DEFINE3(execve)`, same tree, and the `execveat` sibling that shares `do_execveat`:

```
SYSCALL_DEFINE3(execve,
        const char __user *, filename,
        const char __user *const __user *, argv,
        const char __user *const __user *, envp)
{
    return do_execve(getname(filename), argv, envp);
}

SYSCALL_DEFINE5(execveat,
        int, fd, const char __user *, filename,
        const char __user *const __user *, argv,
        const char __user *const __user *, envp,
        int, flags)
{
    int lookup_flags = (flags & AT_EMPTY_PATH) ? LOOKUP_EMPTY : 0;

    return do_execveat(fd,
               getname_flags(filename, lookup_flags, NULL),
               argv, envp, flags);
}
```

`filename`, `argv`, and `envp` arrive as user pointers. They are virtual addresses in the calling task's current `mm`. `do_execveat_common` counts the argument and environment vectors, then `copy_strings` copies the bytes onto the new stack image. The comment on `copy_strings` in this file says it copies argument and environment strings from the old process's memory to the new process's stack. The copy itself uses `copy_from_user`. That is Week 8's rule on a larger buffer: `copy_from_user` translates the user virtual address through the caller's page tables and writes the bytes into pages that will become the new user stack.

`bprm_execve` is the body that opens the file and runs the handler. In `fs/exec.c`, `bprm_execve`, same tree:

```
static int bprm_execve(struct linux_binprm *bprm,
               int fd, struct filename *filename, int flags)
{
    struct file *file;
    int retval;

    /*
     * Cancel any io_uring activity across execve
     */
    io_uring_task_cancel();

    retval = prepare_bprm_creds(bprm);
    if (retval)
        return retval;

    check_unsafe_exec(bprm);
    current->in_execve = 1;

    file = do_open_execat(fd, filename, flags);
    retval = PTR_ERR(file);
    if (IS_ERR(file))
        goto out_unmark;

    sched_exec();

    bprm->file = file;
```

`prepare_bprm_creds` builds the credential object the new image will commit. `do_open_execat` opens the executable and stores the `struct file` on `bprm`. `exec_binprm`, which this function calls next, records `old_pid = current->pid` before it searches for a binary handler, then calls `search_binary_handler`. The comment there says the pid is fetched before `load_binary` changes it. For a single-threaded caller, `de_thread` finds `thread_group_empty` and leaves the task as its own leader. When the caller is a non-leader thread, `de_thread` calls `exchange_tids` and `transfer_pid` so the surviving task takes the previous group's TGID. The process id `getpid` reports — the thread-group id — is the identity that continues. This week's labs are single-threaded, so the same `task_struct` keeps the same `pid` and `tgid` fields while its `mm` is replaced.

On success, `bprm_execve` clears `current->in_execve` and returns. The user instruction that resumes is in the new image. The syscall wrapper in the old image receives a return value only on the failure path.

---

3. `begin_new_exec` is the point of no return

The ELF handler, once it has parsed headers and built a new `mm`, calls `begin_new_exec`. In `fs/exec.c`, `begin_new_exec`, same tree:

```
int begin_new_exec(struct linux_binprm * bprm)
{
    struct task_struct *me = current;
    int retval;

    /* Once we are committed compute the creds */
    retval = bprm_creds_from_file(bprm);
    if (retval)
        return retval;

    /*
     * Ensure all future errors are fatal.
     */
    bprm->point_of_no_return = true;

    /*
     * Make this the only thread in the thread group.
     */
    retval = de_thread(me);
    if (retval)
        goto out;

    /* Ensure the files table is not shared. */
    retval = unshare_files();
    if (retval)
        goto out;

    /*
     * Must be called _before_ exec_mmap() as bprm->mm is
     * not visibile until then. This also enables the update
     * to be lockless.
     */
    set_mm_exe_file(bprm->mm, bprm->file);
```

`point_of_no_return` is why a later mapping error kills the task. `de_thread` makes this task the sole member of the thread group, as section 2 described. `unshare_files` gives the task a private `files_struct` if it had been sharing one under `CLONE_FILES`, so the closes that follow affect this task's table. `set_mm_exe_file` installs the executable `struct file` on the new `mm`. That is the object `/proc/self/exe` will name after the call. Day 1's `proc_lab` reads it back.

`exec_mmap` then publishes the new address space. In `fs/exec.c`, `exec_mmap`, same tree:

```
    task_lock(tsk);
    membarrier_exec_mmap(mm);

    local_irq_disable();
    active_mm = tsk->active_mm;
    tsk->active_mm = mm;
    tsk->mm = mm;
    /*
     * This prevents preemption while active_mm is being loaded and
     * it and mm are being updated, which could cause problems for
     * lazy tlb mm refcounting when these are updated by context
     * switches. Not all architectures can handle irqs off over
     * activate_mm yet.
     */
    if (!IS_ENABLED(CONFIG_ARCH_WANT_IRQS_OFF_ACTIVATE_MM))
        local_irq_enable();
    activate_mm(active_mm, mm);
```

`tsk->mm` and `tsk->active_mm` swing to the new descriptor, and `activate_mm` loads its `pgd` into the core's page-table root. The old `mm` is released with `mmput` on the success path just below this excerpt. The task pointer, the pid, and the kernel stack of this syscall stay. The user translations change underneath the still-running kernel path. Week 5's walk, from the next user instruction on, starts at the new root. ToyOS `address_space_activate` is the one-process form of `activate_mm`.

Still in `begin_new_exec`, after the old mappings are released, the function makes the signal table private, drops thread state, and applies close-on-exec:

```
    /*
     * Make the signal table private.
     */
    retval = unshare_sighand(me);
    if (retval)
        goto out_unlock;

    /*
     * Ensure that the uaccess routines can actually operate on userspace
     * pointers:
     */
    force_uaccess_begin();

    me->flags &= ~(PF_RANDOMIZE | PF_FORKNOEXEC | PF_KTHREAD |
                   PF_NOFREEZE | PF_NO_SETAFFINITY);
    flush_thread();
    me->personality &= ~bprm->per_clear;

    /*
     * We have to apply CLOEXEC before we change whether the process is
     * dumpable (in setup_new_exec) to avoid a race with a process in userspace
     * trying to access the should-be-closed file descriptors of a process
     * undergoing exec(2).
     */
    do_close_on_exec(me->files);
```

`flush_thread` clears architecture-specific user thread state. The `PF_FORKNOEXEC` flag coming off is the kernel's record that this task has now exec'd. `do_close_on_exec` runs before the image is entered, so the new program's descriptor table already lacks the close-on-exec indices.

The disposition update and the credential commit are the next committed steps. Same function:

```
    __set_task_comm(me, kbasename(bprm->filename), true);

    /* An exec changes our domain. We are no longer part of the thread
       group */
    WRITE_ONCE(me->self_exec_id, me->self_exec_id + 1);
    flush_signal_handlers(me, 0);

    /*
     * install the new credentials for this executable
     */
    security_bprm_committing_creds(bprm);

    commit_creds(bprm->cred);
```

`__set_task_comm` stores the new short name on the task. Day 1's `comm` field is that array. `flush_signal_handlers(me, 0)` rewrites dispositions. `commit_creds` installs the prepared `struct cred` into the task's credential pointers. The task object is the same pointer `current` was at the start of the syscall.

---

4. Caught handlers become `SIG_DFL`; `SIG_IGN` stays

In `kernel/signal.c`, `flush_signal_handlers`, same tree:

```
void
flush_signal_handlers(struct task_struct *t, int force_default)
{
    int i;
    struct k_sigaction *ka = &t->sighand->action[0];
    for (i = _NSIG ; i != 0 ; i--) {
        if (force_default || ka->sa.sa_handler != SIG_IGN)
            ka->sa.sa_handler = SIG_DFL;
        ka->sa.sa_flags = 0;
#ifdef __ARCH_HAS_SA_RESTORER
        ka->sa.sa_restorer = NULL;
#endif
        sigemptyset(&ka->sa.sa_mask);
        ka++;
    }
}
```

`begin_new_exec` passes `force_default` as 0. The handler is assigned `SIG_DFL` when the current handler is anything other than `SIG_IGN`. A slot that already holds `SIG_IGN` keeps `SIG_IGN`. A caught handler — a user function pointer — becomes `SIG_DFL`. A slot that was already `SIG_DFL` stays `SIG_DFL`. Every slot, including the ignored ones, has `sa_flags` cleared, the restorer cleared on architectures that have one, and `sa_mask` emptied. That is the disposition rule this tree implements at exec. The application contract for the rest of signal state across exec is POSIX; the function that performs the replacement of the disposition table is this one, called from `begin_new_exec` after `unshare_sighand`.

The new program therefore starts with default action for every signal the old program had caught, and with ignore still installed for every signal the old program had set to `SIG_IGN`. Installing a handler is the new image's job, after it begins running.

---

5. `FD_CLOEXEC` closes one descriptor index

Day 2's `dup_fd` copied the `close_on_exec` bitmap into the child's private `files_struct`. The bit lives beside the descriptor index. `struct file` holds `f_pos` and `f_count`. The close-on-exec bit lives in the fdtable.

In `fs/file.c`, `do_close_on_exec`, same tree:

```
void do_close_on_exec(struct files_struct *files)
{
    unsigned i;
    struct fdtable *fdt;

    /* exec unshares first */
    spin_lock(&files->file_lock);
    for (i = 0; ; i++) {
        unsigned long set;
        unsigned fd = i * BITS_PER_LONG;
        fdt = files_fdtable(files);
        if (fd >= fdt->max_fds)
            break;
        set = fdt->close_on_exec[i];
        if (!set)
            continue;
        fdt->close_on_exec[i] = 0;
        for ( ; set ; fd++, set >>= 1) {
            struct file *file;
            if (!(set & 1))
                continue;
            file = fdt->fd[fd];
            if (!file)
                continue;
            rcu_assign_pointer(fdt->fd[fd], NULL);
            __put_unused_fd(files, fd);
            spin_unlock(&files->file_lock);
            filp_close(file, files);
            cond_resched();
            spin_lock(&files->file_lock);
        }
```

The comment matches `begin_new_exec`: exec unshares the file table first. The loop walks the bitmap, clears it, and `filp_close`s each marked descriptor. `filp_close` drops that reference on the `struct file`. If the parent, or another task, still holds a reference from its own table, the open-file description remains, with its `f_pos`. The new image's index is gone. Descriptors whose close-on-exec bit is clear stay in the table, with the same `struct file` pointer the child inherited at `fork`. That is how a shell passes a pipe end to a command, and how it keeps a password file from being passed: the bit, set on the index before `exec`, makes `do_close_on_exec` drop it.

Standard descriptors 0, 1, and 2 usually have the bit clear, so the new image inherits the terminal or the pipe the shell arranged. The offset and status flags are the shared description from Day 2. The index numbers match because `dup_fd` copied them and this loop left them in place.

---

6. The ELF handler maps `PT_LOAD` and chooses an entry

Week 3's program headers are what `load_elf_binary` reads. In `fs/binfmt_elf.c`, `load_elf_binary`, same tree, after the locals that name `elf_entry`, `e_entry`, and the program-header cursor, the handler checks the image and scans for `PT_INTERP`:

```
    retval = -ENOEXEC;
    /* First of all, some simple consistency checks */
    if (memcmp(elf_ex->e_ident, ELFMAG, SELFMAG) != 0)
        goto out;

    if (elf_ex->e_type != ET_EXEC && elf_ex->e_type != ET_DYN)
        goto out;
    if (!elf_check_arch(elf_ex))
        goto out;
    if (elf_check_fdpic(elf_ex))
        goto out;
    if (!bprm->file->f_op->mmap)
        goto out;

    elf_phdata = load_elf_phdrs(elf_ex, bprm->file);
    if (!elf_phdata)
        goto out;

    elf_ppnt = elf_phdata;
    for (i = 0; i < elf_ex->e_phnum; i++, elf_ppnt++) {
        char *elf_interpreter;

        if (elf_ppnt->p_type == PT_GNU_PROPERTY) {
            elf_property_phdata = elf_ppnt;
            continue;
        }

        if (elf_ppnt->p_type != PT_INTERP)
            continue;
```

`ELFMAG` is the four magic bytes ToyOS checks as `ELF_MAGIC`. `ET_EXEC` and `ET_DYN` are the types this function maps. `elf_check_arch` requires `e_machine` to match this kernel. ToyOS hard-codes machine 62. When the loop finds `PT_INTERP`, it reads that pathname and opens the interpreter. It then walks `PT_LOAD` headers. `make_prot` turns `PF_R`, `PF_W`, and `PF_X` into `PROT_READ`, `PROT_WRITE`, and `PROT_EXEC`. Each loadable segment is mapped with `elf_map` at `p_vaddr` plus a load bias. For `ET_DYN` the bias is the randomized placement of the image. For a fixed `ET_EXEC` the bias is zero and the map is `MAP_FIXED`. Where `p_memsz` exceeds `p_filesz`, `set_brk` and `padzero` cover the zero-filled tail. That is Week 3's `.bss` rule, performed with anonymous pages.

After the executable's segments are mapped, the handler selects the user entry. Same function:

```
    if (interpreter) {
        elf_entry = load_elf_interp(interp_elf_ex,
                        interpreter,
                        load_bias, interp_elf_phdata,
                        &arch_state);
        if (!IS_ERR((void *)elf_entry)) {
            /*
             * load_elf_interp() returns relocation
             * adjustment
             */
            interp_load_addr = elf_entry;
            elf_entry += interp_elf_ex->e_entry;
        }
        if (BAD_ADDR(elf_entry)) {
            retval = IS_ERR((void *)elf_entry) ?
                    (int)elf_entry : -EINVAL;
            goto out_free_dentry;
        }
        reloc_func_desc = interp_load_addr;

        exe_file_allow_write_access(interpreter);
        fput(interpreter);

        kfree(interp_elf_ex);
        kfree(interp_elf_phdata);
    } else {
        elf_entry = e_entry;
        if (BAD_ADDR(elf_entry)) {
            retval = -EINVAL;
            goto out_free_dentry;
        }
    }
```

`e_entry` was set to `elf_ex->e_entry + load_bias`, the executable's own entry in the placed image. If `PT_INTERP` was present, `elf_entry` becomes the interpreter's placed entry, and the executable's entry is preserved for the auxiliary vector. The kernel will enter user mode at `elf_entry`. The dynamic linker, running in user mode, is what eventually transfers to the executable's entry. Week 3 named those stages. This function is the kernel half: it maps both ELFs and chooses which entry register to load.

With no interpreter, `elf_entry` is the executable's `e_entry` plus bias, which is the ToyOS path: map `PT_LOAD`, jump to the header entry.

---

7. The initial stack carries `argc`, `argv`, `envp`, and auxv

`copy_strings` has already placed the argument and environment bytes at the high end of the new stack, walking downward. `create_elf_tables` builds the pointer vector the ABI requires below them. In `fs/binfmt_elf.c`, `create_elf_tables`, same tree, the auxiliary vector is assembled in `mm->saved_auxv` and then copied to user memory:

```
    NEW_AUX_ENT(AT_HWCAP, ELF_HWCAP);
    NEW_AUX_ENT(AT_PAGESZ, ELF_EXEC_PAGESIZE);
    NEW_AUX_ENT(AT_CLKTCK, CLOCKS_PER_SEC);
    NEW_AUX_ENT(AT_PHDR, phdr_addr);
    NEW_AUX_ENT(AT_PHENT, sizeof(struct elf_phdr));
    NEW_AUX_ENT(AT_PHNUM, exec->e_phnum);
    NEW_AUX_ENT(AT_BASE, interp_load_addr);
    NEW_AUX_ENT(AT_FLAGS, 0);
    NEW_AUX_ENT(AT_ENTRY, e_entry);
    NEW_AUX_ENT(AT_UID, from_kuid_munged(cred->user_ns, cred->uid));
    NEW_AUX_ENT(AT_EUID, from_kuid_munged(cred->user_ns, cred->euid));
    NEW_AUX_ENT(AT_GID, from_kgid_munged(cred->user_ns, cred->gid));
    NEW_AUX_ENT(AT_EGID, from_kgid_munged(cred->user_ns, cred->egid));
    NEW_AUX_ENT(AT_SECURE, bprm->secureexec);
    NEW_AUX_ENT(AT_RANDOM, (elf_addr_t)(unsigned long)u_rand_bytes);
#ifdef ELF_HWCAP2
    NEW_AUX_ENT(AT_HWCAP2, ELF_HWCAP2);
#endif
    NEW_AUX_ENT(AT_EXECFN, bprm->exec);
```

`AT_ENTRY` is the executable's `e_entry` plus load bias, even when the first user instruction is the interpreter. `AT_BASE` is the interpreter's load address, zero when there is no interpreter. `AT_PHDR` points at the program headers mapped into the new image. `AT_PAGESZ` is the page size the Week 5 walk uses. `AT_UID` and the other ids are taken from the credential object `commit_creds` is installing. `AT_RANDOM` is sixteen bytes the kernel generated for the user stack. Further entries in the same function, when the architecture or the call provides them, add `AT_PLATFORM`, `AT_BASE_PLATFORM`, and `AT_EXECFD`. The vector ends with `AT_NULL`.

The word vector sits at the low end of this stack block. `put_user(argc, sp++)` stores the count. A loop then `put_user`s each `argv` pointer, using `strnlen_user` to advance to the next string, and stores a zero word. The same pattern follows for `envp`. `copy_to_user` writes the auxiliary vector from `mm->saved_auxv`. `mm->start_stack` becomes `bprm->p`, the stack pointer the entry code will see.

```
high addresses
┌──────────────────────────────────────────┐
│ argument and environment strings     │  copied by copy_strings
│ AT_RANDOM bytes, platform string     │
├──────────────────────────────────────────┤
│ auxv: AT_* key, value, …, AT_NULL    │
│ envp pointers, then 0                │
│ argv pointers, then 0                │
│ argc                                 │  ← user stack pointer at entry
low addresses
```

This is the stack Week 3's `_start` consumes. The kernel built it. The dynamic linker, libc startup, and `main` are still ahead in user mode. `RIP` at the first user instruction names `elf_entry`. The stack pointer names `argc`.

At the end of `load_elf_binary` the handler takes `current_pt_regs()`, the saved user frame on this task's kernel stack, optionally runs `ELF_PLAT_INIT` for ABIs that seed extra registers, calls `finalize_exec`, and calls `start_thread(regs, elf_entry, bprm->p)`.

On x86-64, `start_thread` in `arch/x86/kernel/process_64.c` passes `__USER_CS` and `__USER_DS` to `start_thread_common`. Same tree:

```
    if (static_cpu_has(X86_BUG_NULL_SEG)) {
        /* Loading zero below won't clear the base. */
        loadsegment(fs, __USER_DS);
        load_gs_index(__USER_DS);
    }

    loadsegment(fs, 0);
    loadsegment(es, _ds);
    loadsegment(ds, _ds);
    load_gs_index(0);

    regs->ip        = new_ip;
    regs->sp        = new_sp;
    regs->cs        = _cs;
    regs->ss        = _ss;
    regs->flags     = X86_EFLAGS_IF;
}

void
start_thread(struct pt_regs *regs, unsigned long new_ip, unsigned long new_sp)
{
    start_thread_common(regs, new_ip, new_sp,
                __USER_CS, __USER_DS, 0);
}
```

The return-to-user path loads `RIP` from `regs->ip` and `RSP` from `regs->sp`, with a user code segment and interrupts unmasked. ToyOS writes the same architectural values into an `iretq` frame and executes `iretq`. The task that returns is the task that entered `execve`.

---

8. Execution trace: the status lab's child becomes `/bin/sh`

`mode_status` in projects/linux-system-programming/src/process_lab.c forks, and the child calls `execl("/bin/sh", "sh", "-c", "exit 7", NULL)`. On failure it `_exit(127)`. The parent `waitpid`s and requires a normal exit of status 7. Illustrated ids below are one run. The wrapper prints the real child pid.

1. `fork` does Day 2's work. The child task exists, `exit_signal` is `SIGCHLD`, and `mm_fork` is a write-protected duplicate of the lab program. `execl` is libc packing `argv` and calling `execve`.
2. `SYSCALL_DEFINE3(execve)` runs in task C. `copy_strings` pulls `"sh"`, `"-c"`, and `"exit 7"` from C's user memory with `copy_from_user`. `bprm_execve` opens `/bin/sh` and sets `current->in_execve`.
3. `load_elf_binary` checks `ELFMAG`, accepts `ET_DYN` or `ET_EXEC`, and checks `e_machine`. `/bin/sh` on a typical Linux host is a dynamic ELF, so the `PT_INTERP` loop opens the dynamic linker. `PT_LOAD` segments of both files are mapped into a new `mm`. `e_entry` in the auxv will name the shell's entry. `elf_entry` names the interpreter's entry.
4. `begin_new_exec` sets the point of no return, `de_thread` sees an empty thread group, `exec_mmap` assigns `C->mm = C->active_mm =` the new mm and `activate_mm` loads the new `pgd`. `mmput` drops `mm_fork`. The COW frames unique to the child become free when their last reference goes. The parent's `mm_p` is a different object and stays. `do_close_on_exec`

closes any index whose bitmap bit is set. The child's standard descriptors, bit clear, remain. `flush_signal_handlers(C, 0)` sets caught handlers to `SIG_DFL` and leaves `SIG_IGN` in place. `commit_creds` installs the credentials prepared for this file. `set_mm_exe_file` makes `/proc/C/exe` name `/bin/sh`.

5. `create_elf_tables` writes `argc == 3`, the three `argv` pointers, a zero, the environment pointers, and the auxv including `AT_ENTRY` and `AT_BASE`. `start_thread` sets the child's user `RIP` to the interpreter and the user `RSP` to `argc`. The syscall returns to user mode. Task C's pid is still C. Task P is still blocked in `waitpid`.
6. The dynamic linker, in user mode, maps the shell's libraries, applies relocations, and transfers to the shell's entry. Libc calls the shell's `main`. The shell parses `-c` and runs `exit 7`, which is a user-space `exit` and then an exit-group syscall. The kernel records status 7 on task C and signals the parent with `SIGCHLD` because `exit_signal` was set at `fork`. `waitpid` in P decodes a normal exit of 7. The printed line is `exec_exit=7` with the child's pid and the parent's pid.

The second child in `mode_status` stays the lab image and `pause`s until the parent sends `SIGTERM`. That wait status is a signal termination. Image replacement happened in the child that reached a successful `execve`.

---

9. Architecture lens — x86-64, AArch64, and RISC-V

`execve`, `begin_new_exec`, `load_elf_binary`, and `create_elf_tables` are generic C. The ISA changes three things: which `e_machine` `elf_check_arch` accepts, which register `start_thread` writes as the resume PC, and which page-table root register `activate_mm` loads from `mm->pgd`.

On x86-64 in this tree, `start_thread_common` writes `regs->ip`, `regs->sp`, `__USER_CS`, `__USER_DS`, and `X86_EFLAGS_IF`. On AArch64, `start_thread` in `arch/arm64/include/asm/processor.h` writes `regs->pc`, `regs->sp`, and `regs->pstate = PSR_MODE_EL0t` after clearing the rest of `pt_regs`. On RISC-V, `start_thread` in `arch/riscv/kernel/process.c` writes `regs->epc`, `regs->sp`, and `regs->status = SR_PIE`, then adds the FPU, vector, and XLEN bits this configuration uses. `ELF_PLAT_INIT`, invoked just before `start_thread` when the architecture defines it, seeds any extra entry registers. The initial stack keeps the SysV shape — `argc`, pointer vectors, auxiliary vector — with pointer width following the ELF class. `AT_ENTRY` remains the executable entry. The first user instruction remains `elf_entry`.

ToyOS requires `header->machine` to equal 62 (`ELF_MACHINE_X86_64`) before it maps a segment. `readelf -h` on the lab binary prints that machine for `process_lab`. The child that execs `/bin/sh` loads a second file, with its own machine, `PT_INTERP`, and `e_entry`, into the same task.

---

10. Mastery: Explain · Draw · Observe · Build

The wrapper is evidence after the trace. From the week directory, run `challenges/day-003-exec-elf-inheritance.sh`. It runs `process_lab status` and prints ELF headers of that lab binary.

Explain

- `execve` replaces the user image of the calling task. `exec_mmap` sets `tsk->mm` and `tsk->active_mm` to the new descriptor and activates its `pgd`. For a single-threaded caller, `pid` and `tgid` stay on that `task_struct`. A non-leader thread execs through `de_thread`, which `transfer_pid`s the group's TGID onto the surviving task.
- `load_elf_binary` requires ELF magic, `ET_EXEC` or `ET_DYN`, and `elf_check_arch`. It maps `PT_LOAD`. With `PT_INTERP`, the first user `RIP` is the interpreter's entry, and `AT_ENTRY` still records the executable's entry.
- `create_elf_tables` places `argc`, `argv`, `envp`, and the auxiliary vector on the new stack. `copy_strings` filled the strings with `copy_from_user`. `start_thread` writes the resume `RIP` and stack pointer into `pt_regs`.
- `do_close_on_exec` closes indices whose `close_on_exec` bit is set and drops those `struct file` references. Other indices keep the shared open-file description inherited at `fork`.
- `flush_signal_handlers(current, 0)` assigns `SIG_DFL` when the current handler differs from `SIG_IGN`, leaves `SIG_IGN` in place, and clears `sa_flags` and `sa_mask` on every slot.

Draw

1. Task C before and after `exec_mmap`: same `task_struct`, old `mm` released, new `mm->pgd` installed.
2. The initial stack from section 7, with `AT_ENTRY` and `AT_BASE` labeled for a dynamic `/bin/sh`.
3. The child's fd table across `do_close_on_exec`: one index with the bit set, removed; descriptors 0, 1, and 2 still pointing at the same `struct file` objects as before the call.

Observe

The program prints `exec_exit=7` with child and parent pids, then `signal_exit=SIGTERM`. The first line is a successful exec of the shell. The second line is the paused child terminated by `SIGTERM`. If `strace` exists, the wrapper keeps `execve`, `wait`, `SIGTERM`, and `exit` lines. `readelf` on `process_lab` shows class, type, machine, entry, and any `PT_INTERP` of the program that calls `fork`. The wrapper's LIMIT line records what this run leaves to the source trace: the initial stack, a runtime `FD_CLOEXEC` probe, and loader relocations.

Build

From `mode_status`, name the task that still exists after `execl` returns successfully inside the child, and name the `mm` that `waitpid` is running on in the parent. Then list three fields `begin_new_exec` rewrites on the child (`mm` via `exec_mmap`, `comm`, dispositions via `flush_signal_handlers`) and three it leaves for a single-threaded caller (the task pointer, `pid` / `tgid`, and descriptors whose close-on-exec bit is clear). Check the prediction against the printed pids: the parent pid in `exec_exit=7` is the task that called `fork`, and the child pid is the task that became the shell.

Why? notebook

1. Why does a shell `fork` before `execve`?
2. Why is `begin_new_exec` a point of no return once `exec_mmap` has published the new `mm`?
3. Why is the first user instruction often the interpreter when `AT_ENTRY` names the executable?

4. Why does `copy_strings` use `copy_from_user`?
5. Why is the close-on-exec bit in the `fdtable` bitmap?
6. Why does `flush_signal_handlers` with `force_default == 0` leave `SIG_IGN` unchanged?
7. Why can `/proc/<child>/exe` name `/bin/sh` while the parent's exe still names `process_lab`?
8. Why does ToyOS need no `exec` syscall for its single embedded image?

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Exec Elf Inheritance

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-003-exec-elf-inheritance.sh
```

- View lab source on GitHub

Day 4 — Wait, Zombies, and Reparenting

Target time: approximately 3 hours

> The child has finished. Its instructions will never run again. The parent still needs a number: how the child ended. That number has to live somewhere after the child's address space is gone.

Days 1–3 built a live task: identity in `task_struct`, a duplicated context after `fork`, and a replaced ELF image after `exec`. Today follows the task after user execution ends. The kernel keeps a dead task until a parent collects the status, because the exit code is kernel state. The program that produced it has already been dismantled.

The excerpts below come from the local tree `c9b2385d6793`, a 5.10.134-based kernel. Record `uname -r` beside that id before you treat a line number as true of the machine you are running. Function names and call order move between releases. The POSIX wait contract is what a C program may rely on; these functions show one implementation of that contract.

---

1. Why the exit code has to outlive the program

A parent creates a child so that it can later learn what the child did. The interesting fact often arrives at the end: the child called `_exit(23)`, or a fatal signal ended it. If the kernel deleted the task at the instant user execution stopped, that fact would vanish with the task.

The fact also cannot stay in the child's virtual memory. `do_exit` drops the address space before any parent reads the result. After `exit_mm`, there is no user mapping in which a C variable could still hold the code. The parent and the child have separate address spaces from the `fork` on Day 2. The parent cannot simply load the child's stack.

The fact lives in the task, in `exit_code`, and the task stays allocated until someone in the parent relationship consumes it. That retained task is the zombie: a kernel record with a PID, an exit word, and enough accounting for `wait` to copy out. Its user instruction stream is finished.

This is the lifetime rule from Week 3. A value is usable while its storage lives. After `exit_mm`, the child's C objects are gone. The exit word's storage is the task structure, and its lifetime runs until a waiter collects it or the kernel sees that the waiter has declined it.

---

2. Two syscalls put the word in, one family of syscalls takes it out

The C library's `exit` and `_exit` are ordinary functions until they issue a system call. In this tree the two kernel entries sit together. `kernel/exit.c`, tree `c9b2385d6793`, lines 894–908:

```
SYSCALL_DEFINE1(exit, int, error_code)
{
    do_exit((error_code&0xff)<<8);
}

/*
 * Take down every thread in the group.  This is called by fatal signals
 * as well as by sys_exit_group (below).
 */
void
do_group_exit(int exit_code)
{
    struct signal_struct *sig = current->signal;

    BUG_ON(exit_code & 0x80); /* core dumps don't get here */
```

`exit_group` (line 936) shifts the status the same way and calls `do_group_exit`. The shift keeps eight bits of user status in the high byte and leaves the low byte clear for a normal exit. `WEXITSTATUS` shifts that byte back down. `WIFEXITED` is the predicate that checks the low byte first.

`exit` ends the calling thread. `exit_group` records `group_exit_code`, marks `SIGNAL_GROUP_EXIT`, and brings the other threads to `do_exit` as well. A single-threaded lab child has one thread, so either entry reaches the same teardown. The kernel's object is still a task inside a signal group; Week 11 takes up the other threads.

`do_exit` is marked `__noreturn`. The user instruction pointer that entered the syscall is a return address the kernel will not use to resume that program. The kernel stack of this task runs the teardown. When teardown finishes, `do_exit` calls `do_task_dead` (line 881). The task is off the path that fetches user instructions.

Fatal signals arrive at the same function from the other direction. `get_signal`, which Day 5 follows, calls `do_group_exit` with the signal number for a default fatal action. That number occupies the low bits of the exit word. `do_notify_parent` in `kernel/signal.c` reads the shape:

```
    task_cputime(tsk, &utime, &stime);
    info.si_utime = nsec_to_clock_t(utime + tsk->signal->utime);
    info.si_stime = nsec_to_clock_t(stime + tsk->signal->stime);

    info.si_status = tsk->exit_code & 0x7f;
    if (tsk->exit_code & 0x80)
        info.si_code = CLD_DUMPED;
    else if (tsk->exit_code & 0x7f)
        info.si_code = CLD_KILLED;
    else {
        info.si_code = CLD_EXITED;
        info.si_status = tsk->exit_code >> 8;
    }
```

`kernel/signal.c`, `do_notify_parent`, tree `c9b2385d6793`, lines 1956–1968. A clear low seven bits means a normal exit, and the status is the high byte. A nonzero low seven bits means a signal. Bit 7 (`0x80`) marks a core dump. The userspace macros `WIFEXITED`, `WIFSIGNALED`, `WEXITSTATUS`, and `WTERMSIG` decode this layout. They are part of the C library's reading of an ABI word, in the same way Week 2 treated a machine encoding as something the C abstract machine does not spell out by itself.

Use the predicate before the extractor. `WEXITSTATUS` on a signaled status shifts bits that encode a signal number. `WTERMSIG` on a normal exit reads bits that were left clear. `WIFSTOPPED` and `WIFCONTINUED` belong to job control, which Day 6 introduces; they are separate from the zombie word.

---

3. `do_exit` releases the user program and keeps the task

`kernel/exit.c`, function `do_exit`, tree `c9b2385d6793`, lines 802–833:

```
    tsk->exit_code = code;
    taskstats_exit(tsk, group_dead);

#ifdef CONFIG_TEXT_UNEVICTABLE
    clean_task_unevict_size(tsk);
#endif
    exit_mm();

    if (group_dead)
        acct_process();
    trace_sched_process_exit(tsk);

    exit_sem(tsk);
    exit_shm(tsk);
    exit_files(tsk);
    exit_fs(tsk);
    if (group_dead)
        disassociate_ctty(1);
    exit_task_namespaces(tsk);
    exit_task_work(tsk);
    exit_thread(tsk);

    /*
     * Flush inherited counters to the parent - before the parent
     * gets woken up by child-exit notifications.
     *
     * because of cgroup mode, must be called before cgroup_exit()
     */
    perf_event_exit_task(tsk);

    sched_autogroup_exit_task(tsk);
    cgroup_exit(tsk);
```

Read the calls as a demolition order.

`tsk->exit_code = code` stores the result while the task structure still owns it. The calls after it drop resources the dead program must release.

`exit_mm` drops the memory descriptor: page tables, the ELF mappings from Day 3, the stack, the heap, and the copy-on-write pages from Day 2. The ELF file on disk remains a file. The process image was the mapping, and the mapping is gone. A zombie keeps a PID and an exit word after `/proc/PID/exe` and the user mappings are finished.

`exit_files` and `exit_fs` drop the file table and the filesystem context. Week 9's file objects are reference-counted; the task's table was one holder. A parent that still holds its own reference to a shared open-file description keeps that description. The child's table dies with the child.

`disassociate_ctty`, when this is the last thread in the group, separates the task from its controlling terminal. Day 6 uses that fact: a session leader's exit changes the terminal's owner. The comment above `perf_event_exit_task` records this tree's ordering: counters flush before the parent is woken by the child-exit notification.

This excerpt stops before `exit_notify`. The same function calls `exit_notify(tsk, group_dead)` at line 841, after RCU and breakpoint cleanup. Notification is where the task becomes a zombie and where children of this task receive a new parent. Resource teardown and parent notification run in that order, as two steps of one `__noreturn` function. Earlier, lines 734–788, `do_exit` panics if it is entered from interrupt context, if `tsk->pid` is zero (the idle task), or if global init is the task exiting. The lab stays on ordinary children.

---

4. `exit_notify` publishes the zombie

`kernel/exit.c`, function `exit_notify`, tree `c9b2385d6793`, lines 654–683:

```
static void exit_notify(struct task_struct *tsk, int group_dead)
{
    bool autoreap;
    struct task_struct *p, *n;
    LIST_HEAD(dead);

    write_lock_irq(&tasklist_lock);
    forget_original_parent(tsk, &dead);

    if (group_dead)
        kill_orphaned_pgrp(tsk->group_leader, NULL);

    tsk->exit_state = EXIT_ZOMBIE;
    if (unlikely(tsk->ptrace)) {
        int sig = thread_group_leader(tsk) &&
                thread_group_empty(tsk) &&
                !ptrace_reparented(tsk) ?
            tsk->exit_signal : SIGCHLD;
        autoreap = do_notify_parent(tsk, sig);
    } else if (thread_group_leader(tsk)) {
        autoreap = thread_group_empty(tsk) &&
            do_notify_parent(tsk, tsk->exit_signal);
    } else {
        autoreap = true;
    }

    if (autoreap) {
        tsk->exit_state = EXIT_DEAD;
        list_add(&tsk->ptrace_entry, &dead);
    }
```

Hold the order. First the dying task's children are reparented (`forget_original_parent`). Then, if the whole thread group is dead, `kill_orphaned_pgrp` may signal a process group that has lost its connection to the session. Day 6 explains that signal. Then the task's `exit_state` becomes `EXIT_ZOMBIE`.

`EXIT_ZOMBIE` is `0x0020` in `include/linux/sched.h` in this tree. The procfs state table in `fs/proc/array.c` prints that report bit as `Z (zombie)`. Day 7 reads that string. The letter `Z` is a projection of `exit_state`, produced when someone reads `/proc`. The authoritative transition is this assignment under `tasklist_lock`.

`do_notify_parent` wakes a parent blocked in `wait` and delivers `SIGCHLD` according to the parent's disposition. `autoreap` comes back true when the parent has arranged to discard the status: `SIGCHLD` ignored, or `SA_NOCLDWAIT`, as the comment in `do_notify_parent` (lines 1972–1979) describes. In that case `exit_state` moves straight to `EXIT_DEAD` and the task is queued for `release_task`. The zombie is the state in which a parent who still wants the word can find it. A parent who has declined the word lets the kernel release the task inside this function.

A non-leader thread takes the `else` branch and autoreaps. The thread-group leader is the task `wait` reports for the process. Sub-threads fold their departure into the group. That is why a zombie you can `wait` for is a group leader whose exit state is still `EXIT_ZOMBIE`.

`release_task` (line 186) is the later free: credentials, signal state, PID links, and eventually the task structure. Until it runs, the PID still names this task. After it runs, the PID may be recycled. A number you wrote down earlier can then name a different task. The lab treats the PID as valid only between the observation and the `waitpid` that consumes it.

---

5. `wait4` copies the word into the parent's address space

The parent learns the word through `wait4`. In this tree libc's `waitpid` is expected to call it: the comment above `SYSCALL_DEFINE3(waitpid)` says `sys_waitpid` remains for compatibility.

`kernel/exit.c`, `SYSCALL_DEFINE4(wait4)`, tree `c9b2385d6793`, lines 1659–1670:

```
SYSCALL_DEFINE4(wait4, pid_t, upid, int __user *, stat_addr,
        int, options, struct rusage __user *, ru)
{
    struct rusage r;
    long err = kernel_wait4(upid, stat_addr, options, ru ? &r : NULL);

    if (err > 0) {
        if (ru && copy_to_user(ru, &r, sizeof(struct rusage)))
            return -EFAULT;
    }
    return err;
}
```

`kernel_wait4` (line 1600) turns the pid argument into a pid type. A positive pid selects `PIDTYPE_PID`. Zero and a negative pid select a process group (`PIDTYPE_PGID`), which is how a shell waits for every member of a job. `-1` selects `PIDTYPE_MAX`, every eligible child. The function then fills a `wait_opts` and calls `do_wait`.

The status crosses back to user memory inside `kernel_wait4`. `kernel/exit.c`, lines 1629–1640:

```
    wo.wo_type  = type;
    wo.wo_pid   = pid;
    wo.wo_flags = options | WEXITED;
    wo.wo_info  = NULL;
    wo.wo_stat  = 0;
    wo.wo_rusage    = ru;
    ret = do_wait(&wo);
    put_pid(pid);
    if (ret > 0 && stat_addr && put_user(wo.wo_stat, stat_addr))
        ret = -EFAULT;

    return ret;
```

`wo.wo_stat` was filled from the zombie in `wait_task_zombie` (lines 1095–1097): the group exit code when `SIGNAL_GROUP_EXIT` is set, and `p->exit_code` otherwise. The same function, before that copy, does a `cmpxchg` of `exit_state` from `EXIT_ZOMBIE` to `EXIT_DEAD` (or `EXIT_TRACE` if a ptracer still needs the task). The comment at lines 1022–1030 says the point: one thread owns the reap. After a successful `cmpxchg`, the status word has been sampled and the task is on the way to `release_task` unless a tracer still holds it.

`put_user` is the Week 8 boundary in production form. The pointer `stat_addr` is a user virtual address in the parent's address space. The kernel checks it and writes one integer. A bad pointer yields `-EFAULT`. After a successful return, `status` is an ordinary C object whose representation is the ABI word from `exit_code`. The syscall return register carries the child's PID, or `-1` / `0` for the error and `WNOHANG` cases. Which register that is depends on the ISA; section 11 names them. The meaning of the PID and the status word is kernel policy.

`waitpid` returning the PID is success. Returning `0` with `WNOHANG` means no selected child has changed state yet. Returning `-1` with `ECHILD` means there is no child matching the request. The lab's injected failure waits once, consumes the only child, waits again, and requires `ECHILD`. `EINTR` means a signal arrived while the parent slept; Day 5 is the event model for that retry.

---

6. Execution trace: exit 23, the letter Z, then the reap

This is the zombie half of `challenges/day-004-wait-zombies-orphans.sh`. The wrapper builds the canonical project and runs the bounded child. PIDs change every run. The sequence of kernel states does not.



Walk it once with the objects named.

1. `fork` creates a second task. Day 2's rule still holds: separate descriptor tables, shared open-file descriptions, copy-on-write pages.
2. The child reaches `_exit(23)`. The syscall argument registers are saved on the kernel entry frame and then left behind: this user context will not resume.
3. `do_exit` stores `(23 & 0xff) << 8` in `exit_code` and `exit_mm` drops the child's page tables.
4. `exit_notify` sets `EXIT_ZOMBIE` and notifies the parent. A parent blocked in `wait` wakes. A parent polling `/proc` can observe `Z`.
5. The lab polls only a child it created, for at most 100 one-millisecond delays, then calls `waitpid`. The poll glimpses kernel state. The `waitpid` consumes it.
6. `wait_task_zombie` wins the `cmpxchg`, and `put_user` writes the word into the parent's `status` object, in the parent's address space.
7. The parent applies `WIFEXITED` and then `WEXITSTATUS` and checks 23.
8. `release_task` drops the last kernel record. After that, the PID may be reused for a different task.

The signal-termination case in the same wrapper follows the same skeleton with a different exit word: low bits hold the signal number, `WIFSIGNALED` is the predicate, and `WTERMSIG` is the extractor. Normal exit 7 and a signaled death stay distinguishable because they set different bits in the same word.

---

7. Reparenting: the parent can disappear first

A zombie is a task with an uncollected exit word. An orphan is a task whose `real_parent` has changed because the old parent exited while this task still existed. A child can be reparented while it is still running and later become a zombie of the new parent. A zombie can be reparented too, so the new parent is the one that collects the word.

The kernel does this in `forget_original_parent`, called from `exit_notify` before the dying task is marked `EXIT_ZOMBIE`. The children must already have a live parent by the time this task becomes something a waiter might release.

`kernel/exit.c`, function `forget_original_parent`, tree `c9b2385d6793`, lines 615–648:

```
static void forget_original_parent(struct task_struct *father,
                    struct list_head *dead)
{
    struct task_struct *p, *t, *reaper;

    if (unlikely(!list_empty(&father->ptraced)))
        exit_ptrace(father, dead);

    /* Can drop and reacquire tasklist_lock */
    reaper = find_child_reaper(father, dead);
    if (list_empty(&father->children))
        return;

    reaper = find_new_reaper(father, reaper);
    list_for_each_entry(p, &father->children, sibling) {
        for_each_thread(p, t) {
            RCU_INIT_POINTER(t->real_parent, reaper);
            BUG_ON((!t->ptrace) != (rcu_access_pointer(t->parent) == father));
            if (likely(!t->ptrace))
                t->parent = t->real_parent;
            if (t->pdeath_signal)
                group_send_sig_info(t->pdeath_signal,
                            SEND_SIG_NOINFO, t,
                            PIDTYPE_TGID);
        }
        /*
         * If this is a threaded reparent there is no need to
         * notify anyone anything has happened.
         */
        if (!same_thread_group(reaper, father))
            reparent_leader(father, p, dead);
    }
    list_splice_tail_init(&father->children, &reaper->children);
}
```

`real_parent` is the kinship parent: the task whose `wait` should collect this child in the ordinary case. `parent` can differ while a tracer is attached; Day 7's `ptrace` stops use that split. For an untraced task the two pointers name the same reaper after this loop.

`reparent_leader` (line 586) forces `exit_signal` to `SIGCHLD` so the new parent receives the usual death notification, and if the child is already a zombie it notifies the new parent immediately. A zombie whose original parent died is still a zombie. The wait obligation moved with `real_parent`. The comment at the top of `forget_original_parent` (lines 607–614) states the two jobs together: make a reaper inherit the children, and check whether any process group became orphaned.

`find_child_reaper` (line 510) starts from the pid namespace's `child_reaper`, normally the namespace init. The comment above `find_new_reaper` (lines 541–547) states the preference: another live thread in the same thread group, then the nearest ancestor in this pid namespace that called `prctl` to become a child subreaper, then the init process of the namespace. The walk itself is:

```
static struct task_struct *find_new_reaper(struct task_struct *father,
                        struct task_struct *child_reaper)
{
    struct task_struct *thread, *reaper;

    thread = find_alive_thread(father);
    if (thread)
        return thread;

    if (father->signal->has_child_subreaper) {
        unsigned int ns_level = task_pid(father)->level;
        /*
         * Find the first ->is_child_subreaper ancestor in our pid_ns.
         * We can't check reaper != child_reaper to ensure we do not
         * cross the namespaces, the exiting parent could be injected
         * by setns() + fork().
         * We check pid->level, this is slightly more efficient than
         * task_active_pid_ns(reaper) != task_active_pid_ns(father).
         */
        for (reaper = father->real_parent;
             task_pid(reaper)->level == ns_level;
             reaper = reaper->real_parent) {
            if (reaper == &init_task)
                break;
            if (!reaper->signal->is_child_subreaper)
                continue;
            thread = find_alive_thread(reaper);
            if (thread)
                return thread;
        }
```

`kernel/exit.c`, `find_new_reaper`, tree `c9b2385d6793`, lines 548–577. The quote stops at the end of the `for` loop. After that walk, the same function returns `child_reaper`.

Step 2 is why "the new parent is PID 1" is a weak observation inside a container or a service manager. The pid namespace's init is PID 1 *in that namespace*. An ancestor that set the subreaper flag is preferred while the walk stays inside the same pid level. The lab therefore creates its own subreaper and checks adoption against that process, which is a fact the program controls.

---

8. The subreaper flag is a prctl on the signal struct

`kernel/sys.c`, inside the `prctl` switch, tree `c9b2385d6793`, lines 2445–2458:

```
    case PR_GET_TID_ADDRESS:
        error = prctl_get_tid_address(me, (int __user * __user *)arg2);
        break;
    case PR_SET_CHILD_SUBREAPER:
        me->signal->is_child_subreaper = !!arg2;
        if (!arg2)
            break;

        walk_process_tree(me, propagate_has_child_subreaper, NULL);
        break;
    case PR_GET_CHILD_SUBREAPER:
        error = put_user(me->signal->is_child_subreaper,
                 (int __user *)arg2);
        break;
```

`is_child_subreaper` marks this task as willing to adopt descendants. `has_child_subreaper` marks descendants so `find_new_reaper` knows a walk is worth doing. `fork.c` in this tree copies `has_child_subreaper` onto a new child from the parent's flag (lines 2376–2381). The propagate walk updates existing descendants when the flag is turned on.

The lab calls `prctl(PR_SET_CHILD_SUBREAPER, 1)` in the top process, then creates a middle child and a grandchild. A readiness pipe makes the grandchild record the middle PID before the middle exits. The grandchild waits, boundedly, until its parent PID changes, reports the before and after IDs, and exits. The top process waits for both descendants and checks that it became the grandchild's parent.

```
top (subreaper)
  │  prctl(PR_SET_CHILD_SUBREAPER, 1)
  │
  ├── middle
  │     └── grandchild
  │           records middle's PID through the pipe
  │
  middle exits
      forget_original_parent
      find_new_reaper sees has_child_subreaper
      walks to top, which has is_child_subreaper
      grandchild->real_parent = top
  │
  grandchild observes the new PPID, writes the report, exits
  top waitpid's both, and both exit words are consumed
```

That adoption is Linux-specific. The portable C and POSIX surface is `waitpid` and the status macros. `PR_SET_CHILD_SUBREAPER` is the tool this lab uses so the experiment has a parent it can name without depending on whatever process happens to be PID 1 in the surrounding namespace.

Reparenting keeps the orphan running. It supplies a task that is allowed to collect a future exit word. It is a parent-pointer update under `tasklist_lock`, followed by a list move of `children` onto the reaper.

---

9. Zombie and orphan side by side

question	zombie	orphan
what changed?	this task's user execution has ended	this task's real_parent
where is the evidence?	exit_state == EXIT_ZOMBIE procfs letter Z	real_parent pointer procfs PPid
is the task runnable?	no user context remains	yes, if it has not itself exited
who acts next?	a waiter consumes exit_code	the new parent, later, if this task exits
what does the lab check?	Z, then waitpid, exit 23	PPID moves from middle to the subreaper

A running grandchild is an orphan after the middle exits, and a zombie only after the grandchild itself exits. The exit-23 child is a zombie of its original parent while that parent is still polling. A zombie whose parent exits before `wait` is reparented and still holds the exit word until the new parent collects it.

The visible zombie window exists because the parent delays `wait`. The child has already stored the word and lost its address space. The operation that removes the zombie is a successful wait, or the autoreap path inside `exit_notify` when the parent has declined `SIGCHLD`.

---

10. How the earlier weeks sit inside this path

Week 1 treats an exception as the CPU leaving the current instruction stream so privileged code can run. `exit` is that departure by request. A fatal signal is the kernel ending the user stream. In both cases the user registers are captured on a kernel frame and then dropped as a resume context. User code supplies `exit_code` only as the syscall argument.

`exit_mm` drops the child's page tables. Copy-on-write pages from Day 2 release when their map count drops. The parent's `status` integer is a store into the parent's mappings. The ELF file stays on disk; the mapping Day 3 created is what disappears. The PID names the task. The path names the file.

`_exit` ends the abstract machine for the child. The library `exit` runs `atexit` handlers before the group-exit syscall; `_exit` goes straight to the syscall, so the lab's status word matches the integer the child passed. After `waitpid`, `status` is an ordinary `int` and the `WIF*` macros read the ABI layout. `fork` created the wait obligation. `exec` kept the PID, so the word describes the image that is current at termination.

ToyOS, Weeks 6–9, has one address space and returns with `iretq` (Week 6). A fault frame exists so the interrupted context can resume. There is no second task holding an `exit_code`. Week 8's syscall path is the entry shape of `exit` and `wait4`. Week 9's file objects are what `exit_files` releases. The adoption report is a pipe: close the unused end in each process, because a leftover write end withholds EOF. xv6 stores the status in the process-table slot, marks the slot a zombie, and has the parent's `wait` copy it out. An orphan's parent pointer moves to init. Read a chosen xv6 revision for the functions. Line numbers in this lesson belong to Linux `c9b2385d6793`.

---

11. Architecture lens

Zombies, reparenting, subreapers, and the exit-word layout are kernel policy shared by x86-64, AArch64, and RISC-V in this tree: `do_exit`, `exit_notify`, `forget_original_parent`, and `wait4`. The ISA changes the syscall instruction (`syscall`, `svc`, `ecall`) and the return register for the PID (`rax`, `x0`, or `a0`). Signal delivery on Day 5 is where the ISA shapes a user-stack frame. Process exit discards the user context and builds no such frame.

---

12. What the wrapper is allowed to touch

Run the day from the Week 10 directory through `../challenges/day-004-wait-zombies-orphans.sh`. The wrapper resolves the canonical project, builds with the project's Makefile, and runs the checked modes. It runs without root and without a network. It creates only descendants of itself, reaps every child it creates, and uses `PR_SET_CHILD_SUBREAPER` so the orphan's new parent is the lab process. The zombie poll is bounded, and `waitpid` follows as soon as `Z` is seen. The lab child's command name is controlled, so its `/proc/PID/stat` scan matches that name; a general parser finds the last `)` of the command field.

Predict three things before you run it: a normal exit and a signal death stay distinguishable after the macros; state `Z` appears and is then consumed with exit code 23; the grandchild's parent PID changes from the middle process to the subreaper. PID values may differ. The decoded status, the state letter, and the adoption relationship are the invariants.

---

13. Mastery: Explain · Draw · Observe · Build

The wrapper is `../challenges/day-004-wait-zombies-orphans.sh`. Passing it is evidence about this tree's lab, and it leaves the explanations below for you to produce. These notes do not record the day as mastered.

Explain

1. Why does `exit_code` have to live in the task structure after `exit_mm`?
2. Why does `sys_exit` shift the user code by eight bits before `do_exit`?
3. What does `exit_notify` set, and when does it move the same task on to `EXIT_DEAD` inside that function?
4. What does `wait_task_zombie`'s `cmpxchg` guarantee about two waiters?
5. Why can a grandchild's new parent be a subreaper other than the namespace init?
6. Which of zombie and orphan describes `exit_state`, and which describes `real_parent`?

Draw

Draw the exit-23 trace from section 6 with your own PID numbers filled in after a run. Mark the moment user virtual memory disappears, the moment `EXIT_ZOMBIE` is stored, and the moment `put_user` writes the parent's `status`. On a second diagram, draw the subreaper tree before and after the middle process exits, with `real_parent` arrows.

Observe

Run the wrapper. Record `uname -r`, the tree id `c9b2385d6793` if you checked the sources, the decoded statuses, the `Z` observation, and the before/after parent IDs. Then open `kernel/exit.c` at `do_exit`, `exit_notify`, `forget_original_parent`, and `wait4` in a tree you have identified, and confirm the names still exist before you quote a line.

Build

In the canonical project, keep the existing reap on every success path. Add a notebook paragraph for the injected double-wait: the first `waitpid` consumes the child, the second returns `-1` with `ECHILD`. Explain which kernel state makes the second call fail. Keep the experiment inside descendants of the lab process, and keep the zombie window bounded by a wait.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Wait Zombies Orphans

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-004-wait-zombies-orphans.sh
```

- View lab source on GitHub

Day 5 — Signals, Masks, and Safe Handlers

Target time: approximately 3 hours

> The program is in the middle of an ordinary function. Something else — a terminal key, another process, or the kernel itself — needs this process to notice an event before that function returns. A signal is how the kernel names that event, holds it until the thread can take it, and enters a user function without the program having called it.

Day 4's exit word is a fact about a task that has already stopped running user code. A signal is a fact about a task that is still running. The handler that runs is ordinary user code: it uses the process's address space, the C calling convention, and the same instruction set as `main`. What is unusual is the entry. The kernel builds a frame so that a later return resumes the interrupted context. That shape is the one Week 1 gave exceptions, and the one ToyOS implements with an interrupt stack and `iretq`. The signal frame and the `rt_sigreturn` path are architectural. The pending set, the blocked set, and the choice of disposition are kernel policy shared across x86-64, AArch64, and RISC-V.

The excerpts are from local tree `c9b2385d6793`, a 5.10.134-based kernel. Record `uname -r` next to that id. Names and line numbers move. The application contract is `sigaction(2)` and the async-signal-safe list, and these functions are one kernel's implementation of it.

---

1. Why the mechanism exists

A process spends almost all of its time in user code or in a syscall the user code asked for. The scheduler, a terminal driver, a parent calling `kill`, and a fault in this process itself can all need attention at a moment the C abstract machine thinks of as "between two sequence points inside a function the programmer wrote."

The hardware already has a way to leave the current instruction stream: the exception and interrupt path from Week 1. That path enters the kernel. A signal is the process-visible continuation of the same idea. The kernel records an event against the task, and at a safe moment — return to user mode, or a syscall that is willing to be interrupted — it may transfer control to a user function the process installed.

Three stored facts make that possible:

```
disposition     what should happen for this signal number
                (ignore, default action, or a user handler)
                process-wide, in sighand->action[]

blocked mask    which numbers this thread must hold pending
                per thread, in task->blocked

pending set     which numbers have been posted and not yet
                taken: thread-directed in task->pending,
                process-directed in signal->shared_pending
```

A blocked signal stays pending. The mask postpones delivery. It leaves the event recorded. Generation, the pending bit, selection of a thread, and delivery are separate steps. A handler running is the delivery step for the "catch" disposition. Default action for a fatal signal is the path into Day 4's `do_group_exit`. Default action for a stop signal is the job-control stop Day 6 depends on.

Traditional signals below `SIGRTMIN` coalesce. `legacy_queue` is true when the number is already in the pending bitmap, and `__send_signal` then leaves the existing entry in place. One pending `SIGUSR1` stands for "at least one `SIGUSR1`." A `sig_atomic_t` flag written by the handler means the same thing: the event happened. Real-time signals, numbered from `SIGRTMIN`, have queueing rules and can carry a `siginfo` payload. Today's lab uses `SIGUSR1` and `SIGUSR2`, which are in the coalescing range.

`SIGKILL` and `SIGSTOP` keep a kernel disposition. The lab's check is that `sigaction` of a `SIGKILL` handler fails with `EINVAL`.

---

2. Posting the signal: `__send_signal`

`kernel/signal.c`, `__send_signal`, tree `c9b2385d6793`, lines 1072–1094:

```
static int __send_signal(int sig, struct kernel_siginfo *info, struct task_struct *t,
            enum pid_type type, bool force)
{
    struct sigpending *pending;
    struct sigqueue *q;
    int override_rlimit;
    int ret = 0, result;

    assert_spin_locked(&t->sighand->siglock);

    result = TRACE_SIGNAL_IGNORED;
    if (!prepare_signal(sig, t, force))
        goto ret;

    pending = (type != PIDTYPE_PID) ? &t->signal->shared_pending : &t->pending;
    /*
     * Short-circuit ignored signals and support queuing
     * exactly one non-rt signal, so that we can get more
     * detailed information about the cause of the signal.
     */
    result = TRACE_SIGNAL_ALREADY_PENDING;
    if (legacy_queue(pending, sig))
        goto ret;
```

The caller holds `sighand->siglock`. `prepare_signal` applies the disposition filters (a signal the process ignores is dropped here, with the special cases `prepare_signal` contains for `SIGCONT` and stop signals). The `type` argument chooses the queue. `PIDTYPE_PID` posts to the target thread's private pending set. Any wider type posts to the thread group's shared pending set, which is how a signal aimed at the process can be taken by whichever thread is eligible.

`legacy_queue` (lines 1067–1069) is `(sig < SIGRTMIN) && sigismember(...)`. A second non-realtime signal, while one is already pending, returns before a new queue entry is allocated. That is the coalesce.

When the signal is new, the function allocates a `sigqueue` for the `siginfo` details, or skips that allocation for `SIGKILL` and for kernel threads. It then sets the bitmap bit and wakes a thread. Lines 1164–1186:

```
out_set:
    signalfd_notify(t, sig);
    sigaddset(&pending->signal, sig);

    /* Let multiprocess signals appear after on-going forks */
    if (type > PIDTYPE_TGID) {
        struct multiprocess_signals *delayed;
        hlist_for_each_entry(delayed, &t->signal->multiprocess, node) {
            sigset_t *signal = &delayed->signal;
            /* Can't queue both a stop and a continue signal */
            if (sig == SIGCONT)
                sigdelsetmask(signal, SIG_KERNEL_STOP_MASK);
            else if (sig_kernel_stop(sig))
                sigdelset(signal, SIGCONT);
            sigaddset(signal, sig);
        }
    }

    complete_signal(sig, t, type);
ret:
    trace_signal_generate(sig, info, t, type != PIDTYPE_PID, result);
    return ret;
}
```

`sigaddset` is the pending bit a later `sigpending` can observe. `complete_signal` chooses a thread to wake. The block under `type > PIDTYPE_TGID` records the signal for forks that are in progress so a signal aimed at a process group is visible to children created during the send. For the lab's `raise` of one thread, `type` is `PIDTYPE_TGID` or `PIDTYPE_PID`, and the private or shared pending bit is the observation.

---

3. Choosing a thread: `complete_signal`

`kernel/signal.c`, `complete_signal`, tree `c9b2385d6793`, lines 991–1025:

```
static void complete_signal(int sig, struct task_struct *p, enum pid_type type)
{
    struct signal_struct *signal = p->signal;
    struct task_struct *t;

    /*
     * Now find a thread we can wake up to take the signal off the queue.
     *
     * If the main thread wants the signal, it gets first crack.
     * Probably the least surprising to the average bear.
     */
    if (wants_signal(sig, p))
        t = p;
    else if ((type == PIDTYPE_PID) || thread_group_empty(p))
        /*
         * There is just one thread and it does not need to be woken.
         * It will dequeue unblocked signals before it runs again.
         */
        return;
    else {
        /*
         * Otherwise try to find a suitable thread.
         */
        t = signal->curr_target;
        while (!wants_signal(sig, t)) {
            t = next_thread(t);
            if (t == signal->curr_target)
                /*
                 * No thread needs to be woken.
                 * Any eligible threads will see
                 * the signal in the queue soon.
                 */
                return;
        }
        signal->curr_target = t;
```

`wants_signal` (lines 974–988) rejects a thread that has the signal blocked, a thread with `PF_EXITING`, and — except for `SIGKILL` — a thread that is stopped or traced or already has a pending signal and is not current. The lab is single-threaded, so the group is empty and the early return applies when the only thread is blocked: the bit stays pending, and the thread will dequeue it when it next returns toward user mode with the mask clear. That is the pending-mode experiment. `raise` posts `SIGUSR1` while the mask blocks it. `sigpending` shows membership. The handler's flag is still clear, because delivery has not run.

When a thread is chosen, the rest of `complete_signal` (lines 1028–1063) either starts a fatal group exit, for a signal whose default action kills the group, or calls `signal_wake_up` so the chosen thread notices `TIF_SIGPENDING` and leaves its sleep. `SIGKILL` wakes even a thread in an uninterruptible section of the wake path. The comment at line 1060 states the ordinary case: the signal is already queued, and the wake tells the thread to dequeue it.

---

4. Delivery: `get_signal` on the way back to user mode

Waking a thread runs the kernel. Entering the handler happens when the architecture's return-to-user path calls `get_signal`. In this tree those callers are:

```
arch/x86/kernel/signal.c      arch_do_signal_or_restart   line 863
arch/arm64/kernel/signal.c    do_signal                    line 847
arch/riscv/kernel/signal.c    do_signal                    line 313
```

Each calls `get_signal(&ksig)` and, when it returns true, `handle_signal`. The shared function starts at `kernel/signal.c` line 2526. It freezes if the system is freezing, runs task work, notifies parents of continued or stopped children, and honors `SIGNAL_GROUP_EXIT` by forcing `SIGKILL`. The dequeue that matters for a caught signal is here, lines 2651–2685:

```
        /*
         * Signals generated by the execution of an instruction
         * need to be delivered before any other pending signals
         * so that the instruction pointer in the signal stack
         * frame points to the faulting instruction.
         */
        signr = dequeue_synchronous_signal(&ksig->info);
        if (!signr)
            signr = dequeue_signal(current, &current->blocked, &ksig->info);

        if (!signr)
            break; /* will return 0 */

        if (unlikely(current->ptrace) && signr != SIGKILL) {
            signr = ptrace_signal(signr, &ksig->info);
            if (!signr)
                continue;
        }

        ka = &sighand->action[signr-1];

        /* Trace actually delivered signals. */
        trace_signal_deliver(signr, &ksig->info, ka);

        if (ka->sa.sa_handler == SIG_IGN) /* Do nothing.  */
            continue;
        if (ka->sa.sa_handler != SIG_DFL) {
            /* Run the handler.  */
            ksig->ka = *ka;

            if (ka->sa.sa_flags & SA_ONESHOT)
                ka->sa.sa_handler = SIG_DFL;

            break; /* will return non-zero "signr" value */
        }
```

`dequeue_signal` skips members of `current->blocked`. A pending, blocked `SIGUSR1` stays in the bitmap through this call, `get_signal` returns false, and the architecture path returns to the interrupted user context with no handler. After `sigprocmask` clears the block, the next return-to-user observes the bit, dequeues it, finds a handler other than `SIG_DFL` and `SIG_IGN`, and returns true. The architecture code then builds the frame.

Synchronous signals — faults produced by an instruction — are dequeued first so the saved instruction pointer refers to the faulting instruction. `SIGUSR1` from `raise` is asynchronous relative to the surrounding C, and it takes the ordinary `dequeue_signal` path.

`ptrace_signal` is the hook Day 7's debugger uses: a traced task can stop here and let the tracer rewrite or swallow the number. The lab runs untraced, so the installed `sigaction` is the disposition that wins.

After this excerpt, a default-action signal falls through to ignore, stop, or `do_group_exit` (the fatal path at the `fatal:` label, line 2741). A caught signal breaks out of the loop. `get_signal` then stores `ksig->sig` and returns true (lines 2791–2793). `signal_delivered`, whose comment begins at line 2796, blocks the delivered signal and the extra set in `sa_mask` unless `SA_NODEFER` is set. Nested delivery of the same number waits until the handler returns. That is a kernel mask update, and it is why a handler that re-raises its own signal usually sees the second instance after it returns.

---

5. Sleeping until a signal is pending: `pause`

A thread that has nothing to do until a signal arrives has to sleep in the kernel, in a state the signal wake can leave. This tree still provides the old call.

`kernel/signal.c`, tree `c9b2385d6793`, lines 4504–4528:

```
#ifdef __ARCH_WANT_SYS_PAUSE

SYSCALL_DEFINE0(pause)
{
    while (!signal_pending(current)) {
        __set_current_state(TASK_INTERRUPTIBLE);
        schedule();
    }
    return -ERESTARTNOHAND;
}

#endif

static int sigsuspend(sigset_t *set)
{
    current->saved_sigmask = current->blocked;
    set_current_blocked(set);

    while (!signal_pending(current)) {
        __set_current_state(TASK_INTERRUPTIBLE);
        schedule();
    }
    set_restore_sigmask();
    return -ERESTARTNOHAND;
}
```

`pause` sleeps in `TASK_INTERRUPTIBLE` until `signal_pending` is true, then returns `-ERESTARTNOHAND`. The C library reports that as `EINTR`. The architecture path runs the handler on the way out when the signal is caught. `sigsuspend` stores the old mask, installs the caller's set for the sleep, and `set_restore_sigmask` puts the old mask back. The lab's pending experiment orders the steps with the mask itself: block, post, observe `sigpending`, then unblock.

---

6. The handler is ordinary user code, entered asynchronously

Once the frame is in place, exception return enters the handler with the user stack pointer aimed at that frame. The handler is a C function compiled with the platform calling convention. Section 7 names the register that carries the signal number. The compiler's abstract machine assumes one control path through a function except where the program names a call. A handler is a second entry the compiler did not see at the interrupted point.

If the interrupted function holds a libc lock, a half-updated `malloc` chunk, or `printf`'s buffer, a handler that calls the same function uses objects mid-update. POSIX names the async-signal-safe functions: the ones specified to be callable from a handler because they leave that libc state alone. `printf`, `malloc`, and most of stdio are outside the list. `signal-safety(7)` on the system you run is the list to read.

The lab's "handler ran" flag is a `volatile sig_atomic_t`. That type is the C type specified for an asynchronous store of one value that a later ordinary read can observe. `volatile` makes the store a visible side effect. Week 11's atomics are the tool for cross-thread memory; this flag is the tool for one thread and a handler.

The self-pipe moves the event onto a normal control path. The handler writes one byte to a nonblocking pipe — `write` of that size is async-signal-safe — after saving `errno` and before restoring it, so a failed write leaves the interrupted function's error intact. The byte sits in the kernel pipe buffer. A later `read` on the ordinary path, where `printf` and allocation are in their usual relationship, consumes it. A full pipe returns `EAGAIN` from that write and the handler still returns; the lab sends one `SIGUSR2` and expects one byte. Close the pipe after delivery can no longer observe the descriptors. The single-threaded lab raises, finishes the one delivery, then closes.

---

7. Architecture lens: the frame and `rt_sigreturn`

This is the week's architectural section for signals. Process groups on Day 6 will not need one. Here the kernel has to materialize a user-mode call, and the calling convention, the stack alignment, the syscall instruction that returns from the handler, and the amount to rewind a restarted syscall are properties of the ISA.

x86-64

`struct rt_sigframe` in `arch/x86/include/asm/sigframe.h` (lines 59–64) holds `pretcode`, a `ucontext uc`, and a `siginfo info`, with floating point state after the struct. `__setup_rt_frame` for the 64-bit configuration writes that frame. Tree `c9b2385d6793`, `arch/x86/kernel/signal.c`, lines 466–486:

```
    /* x86-64 should always use SA_RESTORER. */
    if (!(ksig->ka.sa.sa_flags & SA_RESTORER))
        return -EFAULT;

    frame = get_sigframe(&ksig->ka, regs, sizeof(struct rt_sigframe), &fp);
    uc_flags = frame_uc_flags(regs);

    if (!user_access_begin(frame, sizeof(*frame)))
        return -EFAULT;

    /* Create the ucontext.  */
    unsafe_put_user(uc_flags, &frame->uc.uc_flags, Efault);
    unsafe_put_user(0, &frame->uc.uc_link, Efault);
    unsafe_save_altstack(&frame->uc.uc_stack, regs->sp, Efault);

    /* Set up to return from userspace.  If provided, use a stub
       already in userspace.  */
    unsafe_put_user(ksig->ka.sa.sa_restorer, &frame->pretcode, Efault);
    unsafe_put_sigcontext(&frame->uc.uc_mcontext, fp, regs, set, Efault);
    unsafe_put_sigmask(set, frame, Efault);
    user_access_end();
```

`get_sigframe` chooses the user stack, including an alternate signal stack when the disposition asked for one. The saved `sigcontext` is a copy of the interrupted user registers. `pretcode` is the restorer the handler will return to. On x86-64 the kernel requires `SA_RESTORER`: the C library supplies a small user stub, usually in the vDSO, whose job is to invoke `rt_sigreturn`.

Lines 493–504 then aim the live `pt_regs` at the handler:

```
    /* Set up registers for signal handler */
    regs->di = sig;
    /* In case the signal handler was declared without prototypes */
    regs->ax = 0;

    /* This also works for non SA_SIGINFO handlers because they expect the
       next argument after the signal number on the stack. */
    regs->si = (unsigned long)&frame->info;
    regs->dx = (unsigned long)&frame->uc;
    regs->ip = (unsigned long) ksig->ka.sa.sa_handler;

    regs->sp = (unsigned long)frame;
```

`ip` becomes the handler. `sp` becomes the frame, so the `ret` at the end of the handler pops `pretcode` and jumps to the restorer. `di`, `si`, and `dx` are the System V argument registers: signal number, `siginfo` pointer, `ucontext` pointer. When user mode resumes, the CPU fetches the handler as if the program had called it. The call is a kernel edit of the register file.

`SYSCALL_DEFINE0(rt_sigreturn)`, lines 656–679, undoes that edit:

```
SYSCALL_DEFINE0(rt_sigreturn)
{
    struct pt_regs *regs = current_pt_regs();
    struct rt_sigframe __user *frame;
    sigset_t set;
    unsigned long uc_flags;

    frame = (struct rt_sigframe __user *)(regs->sp - sizeof(long));
    if (!access_ok(frame, sizeof(*frame)))
        goto badframe;
    if (__get_user(*(__u64 *)&set, (__u64 __user *)&frame->uc.uc_sigmask))
        goto badframe;
    if (__get_user(uc_flags, &frame->uc.uc_flags))
        goto badframe;

    set_current_blocked(&set);

    if (!restore_sigcontext(regs, &frame->uc.uc_mcontext, uc_flags))
        goto badframe;

    if (restore_altstack(&frame->uc.uc_stack))
        goto badframe;

    return regs->ax;
```

`regs->sp - sizeof(long)` recovers the frame after the restorer's syscall entry has pushed a return slot. `restore_sigcontext` copies the saved user registers back into `pt_regs`. The mask saved at delivery is restored. The next return to user mode resumes the interrupted instruction stream. A bad frame takes `signal_fault` (line 682) and the process receives `SIGSEGV`. Forging a frame from user C is a way to corrupt that restore. The lab proves the handler ran. It leaves the frame bytes on the stack for the kernel and the C library.

The same file's `arch_do_signal_or_restart` (line 863) rewinds a restartable syscall by `regs->ip -= 2`, because the x86-64 `syscall` instruction is two bytes. That constant is an ISA fact. It is why this lesson's frame discussion is architectural and Day 6's process groups are not.

AArch64 and RISC-V

Both architectures build a different frame and return through their own `rt_sigreturn`.

AArch64's `setup_return` in `arch/arm64/kernel/signal.c`, lines 735–764, sets the handler's entry registers and the link register:

```
    regs->regs[0] = usig;
    regs->sp = (unsigned long)user->sigframe;
    regs->regs[29] = (unsigned long)&user->next_frame->fp;
    regs->pc = (unsigned long)ka->sa.sa_handler;

    /*
     * Signal delivery is a (wacky) indirect function call in
     * userspace, so simulate the same setting of BTYPE as a BLR
     * <register containing the signal handler entry point>.
     * Signal delivery to a location in a PROT_BTI guarded page
     * that is not a function entry point will now trigger a
     * SIGILL in userspace.
     *
     * If the signal handler entry point is not in a PROT_BTI
     * guarded page, this is harmless.
     */
    if (system_supports_bti()) {
        regs->pstate &= ~PSR_BTYPE_MASK;
        regs->pstate |= PSR_BTYPE_C;
    }

    /* TCO (Tag Check Override) always cleared for signal handlers */
    regs->pstate &= ~PSR_TCO_BIT;

    if (ka->sa.sa_flags & SA_RESTORER)
        sigtramp = ka->sa.sa_restorer;
    else
        sigtramp = VDSO_SYMBOL(current->mm->context.vdso, sigtramp);

    regs->regs[30] = (unsigned long)sigtramp;
```

`pc` is the handler, `sp` is the frame, `x0` is the signal number, and `x30` is the restorer, so a normal function return branches to `sigreturn` machinery. `SYSCALL_DEFINE0(rt_sigreturn)` is at line 531 of the same file. Branch Target Identification and tag-check override are architectural state this kernel writes because a handler entry has to look like a real indirect call on that CPU. x86-64 has no `PSTATE.BTYPE` field to set. The pending-bit logic in `kernel/signal.c` is unchanged.

RISC-V's `setup_rt_frame` in `arch/riscv/kernel/signal.c`, lines 238–264:

```
    /* Set up to return from userspace. */
#ifdef CONFIG_MMU
    regs->ra = (unsigned long)VDSO_SYMBOL(
        current->mm->context.vdso, rt_sigreturn);
#else
    /*
     * For the nommu case we don't have a VDSO.  Instead we push two
     * instructions to call the rt_sigreturn syscall onto the user stack.
     */
    if (copy_to_user(&frame->sigreturn_code, __user_rt_sigreturn,
             sizeof(frame->sigreturn_code)))
        return -EFAULT;
    regs->ra = (unsigned long)&frame->sigreturn_code;
#endif /* CONFIG_MMU */

    /*
     * Set up registers for signal handler.
     * Registers that we don't modify keep the value they had from
     * user-space at the time we took the signal.
     * We always pass siginfo and mcontext, regardless of SA_SIGINFO,
     * since some things rely on this (e.g. glibc's debug/segfault.c).
     */
    regs->epc = (unsigned long)ksig->ka.sa.sa_handler;
    regs->sp = (unsigned long)frame;
    regs->a0 = ksig->sig;                     /* a0: signal number */
    regs->a1 = (unsigned long)(&frame->info); /* a1: siginfo pointer */
    regs->a2 = (unsigned long)(&frame->uc);   /* a2: ucontext pointer */
```

`epc` is the PC that exception return will resume. `ra` is the return address, aimed at the vDSO `rt_sigreturn` when the kernel has an MMU. `SYSCALL_DEFINE0(rt_sigreturn)` is at line 138 of the same file. `do_signal` (line 313) rewinds a restartable syscall with `regs->epc -= 0x4`, one RISC-V instruction, where x86-64 subtracted 2.

```
                x86-64              AArch64              RISC-V
handler PC      regs->ip            regs->pc             regs->epc
stack           regs->sp            regs->sp             regs->sp
signal number   regs->di            regs->regs[0]        regs->a0
return tramp    frame->pretcode     regs->regs[30]       regs->ra
sigreturn       rt_sigreturn        rt_sigreturn         rt_sigreturn
                (line 656)          (line 531)           (line 138)
syscall rewind  ip -= 2             pc = restart_addr    epc -= 4
```

The shared idea is one sentence: the kernel writes a user frame and a user PC so that exception return enters the handler, and `rt_sigreturn` later copies the saved context back. The registers, the trampoline, and the rewind distance are per ISA.

ToyOS

Week 6 gives a fault a known stack through the IST and returns with `iretq`. The IDT stub builds a frame so the handler can run and so `iretq` can restore the interrupted context. Linux signal delivery is that pattern with the handler in user mode:

```
ToyOS fault                          Linux caught signal
-----------                          -------------------
CPU traps, IST may switch stack      CPU traps or syscall returns
kernel builds a frame                kernel builds a user rt_sigframe
handler runs on a kernel stack       handler runs in the user address space
iretq restores the saved context     rt_sigreturn restores sigcontext
                                     then the kernel returns to user mode
```

ToyOS records the fault with an explicit marker and returns through `iretq` in one address space. Linux adds a `sigaction` table and a per-thread blocked mask, and it returns through `rt_sigreturn`. Week 8's `int 0x80` path is the shape of that return entry: user code crosses with a call number, and the kernel checks the frame (`access_ok`, `__get_user`) before trusting it. Which number is pending, and which thread is eligible, stays in `kernel/signal.c`.

---

8. Execution trace: blocked `SIGUSR1`, then the self-pipe

This is the sequence inside `../challenges/day-005-signals-safe-handlers.sh`. The wrapper runs the canonical modes and sends signals only to the current process or a child it created.

```
install sigaction for SIGUSR1
sigprocmask(SIG_BLOCK) adds SIGUSR1 to current->blocked
raise(SIGUSR1)
    __send_signal
        pending bit set
        legacy_queue will collapse a second SIGUSR1
        complete_signal: the only thread blocks it, so no delivery yet
sigpending shows SIGUSR1
handler flag is still 0

sigprocmask unblocks SIGUSR1
return toward user mode
    arch_do_signal_or_restart / do_signal
        get_signal dequeues SIGUSR1
        disposition is the user handler
        __setup_rt_frame / setup_rt_frame
            save registers into the user frame
            set PC to the handler, SP to the frame
    exception return
handler stores volatile sig_atomic_t
handler returns
    restorer issues rt_sigreturn
    restore_sigcontext copies the saved registers back
interrupted user code continues
flag is 1
```

The self-pipe mode is the same delivery with a different body:

```
SIGUSR2 handler
    save errno
    write one byte to the nonblocking pipe
    restore errno
    return
ordinary read consumes the byte
ordinary code prints the result
```

The checked failures sit beside those traces. `sigaction` for `SIGKILL` returns `EINVAL`. `kill` of a pid the lab chooses so that it names no process returns `ESRCH`. Signal zero asks about permission and existence and posts nothing. `SA_RESTART` is set on the lab disposition; the architecture return path applies it per syscall return code. The lab's evidence is the handler flag and the pipe byte.

Run the wrapper from the Week 10 directory. It builds through the project Makefile, runs without root, and targets the current process or a descendant. Predict that `SIGUSR1` is pending while blocked and that the flag changes after the unblock, and that `SIGUSR2` produces one pipe byte on the normal path.

---

9. Mastery: Explain · Draw · Observe · Build

The wrapper is `../challenges/day-005-signals-safe-handlers.sh`. A passing run shows the pending bit and the pipe byte. The explanations below are still yours to write. These notes leave mastery open.

Explain

1. Where does a blocked signal live between `raise` and the unblock?
2. Why does `legacy_queue` make a second `SIGUSR1` invisible as a separate event?
3. Why is the handler ordinary user code, and why does that fact restrict it to async-signal-safe functions?
4. What does the self-pipe move, and which context is allowed to call `printf`?
5. What does x86-64 store in `pretcode`, and what does `rt_sigreturn` copy back into `pt_regs`?
6. Why do AArch64 and RISC-V have their own `rt_sigreturn` even though `get_signal` is shared?

Draw

Draw the pending-bit trace from section 8, and a second picture of the user stack at the moment `regs->ip` (or `pc`, or `epc`) first points at the handler. Label the saved context, the trampoline, and the interrupted instruction the trampoline will resume. Mark which boxes are ISA-specific.

Observe

Run the wrapper. Record the pending membership before the unblock, the flag afterward, and the pipe byte. Record `uname -r`. If you open `kernel/signal.c` or `arch/<isa>/kernel/signal.c`, confirm the tree id `c9b2385d6793` or say which tree you actually have before quoting a line.

Build

In the canonical program, keep the handler's work on the safe list. Write a short note that names one libc function the handler leaves to the ordinary path, and why the pipe is the handoff. Keep signals aimed at the lab process or its children.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Signals Safe Handlers

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-005-signals-safe-handlers.sh
```

- View lab source on GitHub

Day 6 — Shell Process Groups and Sessions

Target time: approximately 3 hours

> A shell's useful trick is that one keystroke can reach every process in a pipeline. `fork` gives each process a PID. The terminal needs a second name: the job. That name is a process group, inside a session, with one group marked foreground on the controlling terminal.

Days 1–3 created tasks and replaced their ELF images. Day 4 kept an exit word until a parent collected it. Day 5 showed how a signal becomes a user-mode entry. Today those pieces become the loop a shell runs: `fork`, arrange a process group, `exec`, `wait`. The group identity is what lets the terminal deliver `SIGINT`, `SIGTSTP`, and `SIGQUIT` to a job.

The lab observes this process's session and creates one short-lived child group. It leaves `setsid`, `tcsetpgrp`, and an interactive read-eval loop alone, so the terminal you are typing in keeps the foreground group it already has. The model below is the mechanism. The wrapper is a bounded look at it.

Excerpts are from tree `c9b2385d6793`, a 5.10.134-based kernel. Record `uname -r` beside that id. `setpgid` and `setsid` are the POSIX calls a program may rely on. The C functions in `kernel/sys.c` are one implementation, and their line numbers belong to this tree.

---

1. Why a PID is the wrong name for a job

A pipeline such as `producer | consumer` is two tasks. Each has a PID, an address space, and a file table. The user thinks of them as one job: Ctrl-C should interrupt both, Ctrl-Z should stop both, and `fg` should wake both. A signal aimed at one PID misses the other.

The kernel's answer is a process group, a set of tasks that share a process-group id (PGID). A signal can be posted to that set. A session is a set of process groups that share a session id (SID), normally the descendants of one login shell or one terminal session. The controlling terminal remembers one of those groups as the foreground group. Keyboard signals go there.

```
session (SID)
├── foreground process group (PGID = terminal's tpgid)
│     ├── shell, while it is waiting at the prompt
│     └── or the job the shell has placed in front
└── background process groups
      └── jobs the shell started and did not put in front

controlling terminal
    one foreground PGID
    SIGINT  / SIGQUIT / SIGTSTP  → that PGID
    a background read  → SIGTTIN to the reader's group
    a background write, when the terminal mode says so → SIGTTOU
```

Five numbers answer five questions:

```
PID     which task
PPID    which real_parent, Day 4's kinship pointer
PGID    which job
SID     which session
tpgid   which group currently owns the terminal's foreground
```

Ancestry and group membership are different relations. A shell can place siblings, and selected descendants, into one PGID. A child can leave its parent's group by calling `setpgid`. Day 4's orphan is a changed `real_parent`. An orphaned *process group* is a group that has lost its connection to a parent in the same session. `kill_orphaned_pgrp`, below, exists for that second meaning.

---

2. `setpgid` moves a task between groups

A new task inherits its parent's PGID and SID at `fork`. The shell then moves the child, or the child moves itself, before `exec`. Both sides often call `setpgid`, because the parent and the child run concurrently and either may reach the call first. The kernel accepts the call from either side under the rules in this function.

`kernel/sys.c`, `SYSCALL_DEFINE2(setpgid)`, tree `c9b2385d6793`, lines 1026–1053:

```
SYSCALL_DEFINE2(setpgid, pid_t, pid, pid_t, pgid)
{
    struct task_struct *p;
    struct task_struct *group_leader = current->group_leader;
    struct pid *pgrp;
    int err;

    if (!pid)
        pid = task_pid_vnr(group_leader);
    if (!pgid)
        pgid = pid;
    if (pgid < 0)
        return -EINVAL;
    rcu_read_lock();

    /* From this point forward we keep holding onto the tasklist lock
     * so that our parent does not change from under us. -DaveM
     */
    write_lock_irq(&tasklist_lock);

    err = -ESRCH;
    p = find_task_by_vpid(pid);
    if (!p)
        goto out;

    err = -EINVAL;
    if (!thread_group_leader(p))
        goto out;
```

A zero `pid` means the calling thread group. A zero `pgid` means "become a group leader whose PGID equals that pid," which is what the lab's helper child does with `setpgid(0, 0)`. The tasklist lock is held so the parent pointer cannot change during the permission checks. Only a thread group leader carries a PGID; a sibling thread gets `-EINVAL`.

The permission half, lines 1055–1080:

```
    if (same_thread_group(p->real_parent, group_leader)) {
        err = -EPERM;
        if (task_session(p) != task_session(group_leader))
            goto out;
        err = -EACCES;
        if (!(p->flags & PF_FORKNOEXEC))
            goto out;
    } else {
        err = -ESRCH;
        if (p != group_leader)
            goto out;
    }

    err = -EPERM;
    if (p->signal->leader)
        goto out;

    pgrp = task_pid(p);
    if (pgid != pid) {
        struct task_struct *g;

        pgrp = find_vpid(pgid);
        g = pid_task(pgrp, PIDTYPE_PGID);
        if (!g || task_session(g) != task_session(group_leader))
            goto out;
    }
```

Read the branches as the shell's race. If the caller is in the parent's thread group, the target must share the session, and it must still have `PF_FORKNOEXEC` set — the flag `fork` sets and `exec` clears. After the child has exec'd, the parent receives `EACCES` and the window for moving it has closed. If the caller is the target itself, the `else` branch allows the call. A session leader (`signal->leader`) stays put. Joining an existing group requires that group to live in the same session.

The update, after `security_task_setpgid` (lines 1082–1095):

```
    err = security_task_setpgid(p, pgid);
    if (err)
        goto out;

    if (task_pgrp(p) != pgrp)
        change_pid(p, PIDTYPE_PGID, pgrp);

    err = 0;
out:
    /* All paths lead to here, thus we are safe. -DaveM */
    write_unlock_irq(&tasklist_lock);
    rcu_read_unlock();
    return err;
}
```

`change_pid(..., PIDTYPE_PGID, ...)` is the whole effect on the common path. PGID is a pid-namespace identity of type `PIDTYPE_PGID`, stored beside `PIDTYPE_PID` and `PIDTYPE_SID`. It is a kernel object. It is the same object on every ISA this tree builds.

The lab's helper takes the self-move path: the child calls `setpgid(0, 0)`, reports PID, PGID, and SID through a pipe, and the parent checks that PID equals PGID and then waits. That proves a group leader. It leaves the two-sided race, and a multi-process pipeline, as the next thing a shell must get right. A production shell calls `setpgid` in both parent and child and treats "already in the right group" as success.

---

3. `setsid` starts a session and drops the terminal

A new session is how a daemon, or a shell that wants a new terminal, detaches from the session that `fork` inherited. `SYSCALL_DEFINE0(setsid)` at line 1211 of `kernel/sys.c` calls `ksys_setsid`. The work is:

`kernel/sys.c`, `set_special_pids` and `ksys_setsid`, tree `c9b2385d6793`, lines 1167–1199:

```
static void set_special_pids(struct pid *pid)
{
    struct task_struct *curr = current->group_leader;

    if (task_session(curr) != pid)
        change_pid(curr, PIDTYPE_SID, pid);

    if (task_pgrp(curr) != pid)
        change_pid(curr, PIDTYPE_PGID, pid);
}

int ksys_setsid(void)
{
    struct task_struct *group_leader = current->group_leader;
    struct pid *sid = task_pid(group_leader);
    pid_t session = pid_vnr(sid);
    int err = -EPERM;

    write_lock_irq(&tasklist_lock);
    /* Fail if I am already a session leader */
    if (group_leader->signal->leader)
        goto out;

    /* Fail if a process group id already exists that equals the
     * proposed session id.
     */
    if (pid_task(sid, PIDTYPE_PGID))
        goto out;

    group_leader->signal->leader = 1;
    set_special_pids(sid);

    proc_clear_tty(group_leader);
```

The new SID is the caller's own PID. `set_special_pids` points both `PIDTYPE_SID` and `PIDTYPE_PGID` at that pid, so a new session leader is also a process-group leader of a fresh group. `signal->leader` records the session-leader bit that `setpgid` later refuses to move. `proc_clear_tty` drops the controlling terminal. The same function then returns the new session id (line 1201) when the checks passed.

The failure checks are the reason a shell that wants a new session forks twice, or forks once and has the child call `setsid`. The caller must not already be a session leader, and its PID must not already be some group's PGID. The lab's wrapper is already inside a session — often one created by the program that launched the terminal — and it leaves that session in place.

After `setsid`, a session leader may open a terminal and become its controlling process, under the rules of the tty layer. The terminal's foreground PGID starts as this new group. `tcsetpgrp` later changes that foreground id. Those steps are how an interactive shell hands the keyboard to a job and takes it back. They also rearrange the terminal the IDE is using if you run them against it. Today's wrapper stops before that handoff.

Day 4's `do_exit` calls `disassociate_ctty` when the last thread of a group exits. A session leader's exit is what drops the controlling terminal for the session. The association is kernel state on the task, released with the rest of the user-facing resources.

---

4. An orphaned process group is a job-control fact

Day 4's `exit_notify` calls `kill_orphaned_pgrp` when the whole thread group dies, and `reparent_leader` calls it when children move to a new parent. The function's comment names the POSIX rule.

`kernel/exit.c`, `kill_orphaned_pgrp`, tree `c9b2385d6793`, lines 316–345:

```
/*
 * Check to see if any process groups have become orphaned as
 * a result of our exiting, and if they have any stopped jobs,
 * send them a SIGHUP and then a SIGCONT. (POSIX 3.2.2.2)
 */
static void
kill_orphaned_pgrp(struct task_struct *tsk, struct task_struct *parent)
{
    struct pid *pgrp = task_pgrp(tsk);
    struct task_struct *ignored_task = tsk;

    if (!parent)
        /* exit: our father is in a different pgrp than
         * we are and we were the only connection outside.
         */
        parent = tsk->real_parent;
    else
        /* reparent: our child is in a different pgrp than
         * we are, and it was the only connection outside.
         */
        ignored_task = NULL;

    if (task_pgrp(parent) != pgrp &&
        task_session(parent) == task_session(tsk) &&
        will_become_orphaned_pgrp(pgrp, ignored_task) &&
        has_stopped_jobs(pgrp)) {
        __kill_pgrp_info(SIGHUP, SEND_SIG_PRIV, pgrp);
        __kill_pgrp_info(SIGCONT, SEND_SIG_PRIV, pgrp);
    }
}
```

A stopped background job is parked in the kernel, waiting for `SIGCONT` or for someone to move it to the foreground. If every parent who could do that has left the session, the job would wait with no process left that is allowed to foreground it. The kernel then posts `SIGHUP` and `SIGCONT` to the group. `SIGHUP`'s default action terminates. `SIGCONT` wakes the stop so the hangup can be delivered. `has_stopped_jobs` (line 304) looks for `SIGNAL_STOP_STOPPED`.

This is a different word from Day 4's orphan process. An orphan process has a new `real_parent` and keeps running. An orphaned process group may still contain running tasks; the special signal pair is for the stopped ones. `get_signal` on Day 5 already consults the same idea: a stop signal other than `SIGSTOP` is discarded when `is_current_pgrp_orphaned` is true, so a background job in an orphaned group is not stopped by `SIGTSTP` with nobody left to continue it. `SIGSTOP` still stops.

`__kill_pgrp_info` is a process-group send. It is the kernel-internal form of the user rule that a negative pid argument to `kill` names a group. The lab sends no job-control signals and no negative-pid kills. The excerpt is here so the exit path from Day 4 and the signal path from Day 5 meet in one place.

---

5. A shell is a loop, plus group identity

For one external command the shell:

1. parses and expands the line in the shell process;
2. creates pipes and opens redirection targets before any child that must inherit them;
3. `fork`s;
4. in the child, and usually also in the parent, calls `setpgid` so the job has a PGID;
5. if the job is foreground, calls `tcsetpgrp` so the terminal's foreground id is that PGID;
6. in the child, `exec`s the new ELF image, which keeps the PID, the PGID, and the SID;
7. `wait`s, with the group pid when the job has more than one member, and reads the exit word Day 4 decoded;
8. restores the terminal's foreground group to the shell and restores the terminal modes it saved.

Builtins such as `cd` change the shell's own filesystem context. Running them only in a child would change a task that exits immediately, and the shell's cwd would stay where it was. The shell therefore has a split: some commands are function calls in the shell process, and some are images loaded by `exec`.

A pipe has to exist before the children. Each child inherits both ends and closes the end it will not use. If the shell keeps a write end, the consumer's `read` waits for an EOF that `exit_files` in the producer will never complete, because the shell's descriptor is still open. After `dup2` onto the standard stream, the child closes the original pipe end. `FD_CLOEXEC` from Day 3 drops descriptors the new image must not see. `exec` failure is still the child, which reports it through the exit word — the lab maps a missing path to exit status 127 — and the parent waits. The parent distinguishes "exec failed" from "the new program exited" by that status, because a successful `exec` does not return to the old image.

```
shell
  | parse
  | pipe() / open redirections
  | fork
  | setpgid in parent ──────┐
  |                         | child
  |                         | setpgid(0, pgid)
  |                         | dup2, close unused ends
  |                         | execve → new ELF, same PID/PGID/SID
  | tcsetpgrp if foreground |
  | waitpid on the group    |  (exit word via wait4, Day 4)
  | tcsetpgrp back to shell
  | print the status the shell's language defines
```

The status a shell prints is a presentation choice on top of the wait word. A pipeline's reported status might be the last command's status, or the first nonzero status if the shell's `pipefail` option is on. That choice is the shell's. The kernel's contribution is one wait word per task, plus the ability to wait for a PGID because `kernel_wait4` treats a negative pid as `PIDTYPE_PGID` (Day 4, lines 1618–1620).

`SIGCHLD` tells the shell a child changed state. The handler belongs to Day 5's discipline: a `sig_atomic_t` or one byte on a self-pipe, and the job table updated on the ordinary path where locks and `malloc` are consistent. `waitpid` with `WNOHANG` on that path collects the exit word. `WUNTRACED` and `WCONTINUED` report stops and continues, which an interactive job-control shell tracks. The canonical lab checks a normal exit and a signal death. It leaves the stop/continue wait options for a later exercise.

The foreground wait is a loop over the group, because a pipeline has several tasks and any of them can stop. Blocking on one PID misses the sibling that received `SIGTSTP`. The shell's job table is user memory. The PGID is the kernel name that makes one `wait` and one signal cover the members.

---

6. Execution trace: one foreground command, then the lab's child group

Imagine the shell is the foreground group and the user runs a single external program that exits 7. This trace is the mechanism. The wrapper exercises the fork/exec/wait portion and a separate child-led `setpgid`, and it leaves `tcsetpgrp` uncalled.

```
shell PGID=S, foreground
    parse "program"
    fork
        child inherits PGID=S, SID, file table
    parent setpgid(child, child)     child setpgid(0, 0)
        whichever runs first: change_pid(PIDTYPE_PGID)
        the child is now PGID=child, still the same SID
    parent tcsetpgrp(terminal, child)     [a real interactive shell]
        terminal foreground id = child's PGID
    child execve("program")
        Day 3: new mappings, same PID, same PGID, same SID
        PF_FORKNOEXEC clears; further parent setpgid gets EACCES
    program runs, _exit(7)
        Day 4: exit_code = 7<<8, EXIT_ZOMBIE, SIGCHLD
    parent waitpid(-PGID) or waitpid(child)
        put_user of the status word
        WIFEXITED / WEXITSTATUS → 7
    parent tcsetpgrp(terminal, shell)
        foreground group is the shell again
```

The lab's checked status mode follows the middle of that picture: fork, a fixed exec, wait, decode exit 7. The helper follows the `setpgid(0, 0)` line only. It reports the child's ids on a pipe, the parent checks that the child is a group leader, and the parent reaps it. Standard input may or may not be a terminal in the environment that launched the wrapper. `isatty` on stdin answers that question for this run. A non-terminal stdin is a true observation about this process, and the session and process-group fields in `/proc/self/stat` are still meaningful: the process sits in whatever session its parent arranged.

What you record from the wrapper:

```
this process    PID, PPID, PGID, SID, isatty(stdin)
status child    fork → exec → wait → decoded exit 7
helper child    setpgid(0,0) → PID == PGID → parent wait
```

PIDs change every run. The invariant is the relationship: the helper's PGID equals its PID, the status child's exit word decodes as 7, and every child the wrapper created is waited for.

---

7. Where the earlier weeks sit

Registers and the ISA. `setpgid` and `setsid` are ordinary syscalls. The call number and the return register depend on the architecture, as on Day 4. The values they pass — pid, pgid — are integers in the ABI registers. After `change_pid`, the user register file is irrelevant to the new group. There is no signal frame, no `rt_sigreturn`, and no rewind of a syscall instruction in this path. x86-64, AArch64, and RISC-V share `kernel/sys.c` for this policy. Day 5's frame was architectural because the handler entry has to look like a call on that CPU. A process group does not.

Virtual memory and ELF. `exec` replaces mappings and keeps PGID and SID. The new image is a different ELF file in the same job. A shell script versus a binary is a difference in the image Day 3 loaded, and the job-control ids survive either load.

The C abstract machine. The shell is a C program (or a program compiled to the same syscall ABI) looping on library calls. `errno` after `setpgid` is how the shell sees `EACCES`, `EPERM`, and `ESRCH`. The job table is ordinary user memory. Updating it from a `SIGCHLD` handler would meet Day 5's safety rule, so the handler posts a byte and the loop updates the table.

Syscalls. `fork` copies the ids. `setpgid` and `setsid` change them. `execve` keeps them and clears `PF_FORKNOEXEC`. `wait4` with a group pid collects the job. `tcsetpgrp` points the terminal at a group. `kill` with a negative pid posts to a group, which is the user-facing side of `__kill_pgrp_info`.

ToyOS and xv6. ToyOS has one user address space and an `iretq` back to it. There is no second task to put in a group, and no terminal foreground id. xv6's shell is a small loop of `fork`, `exec`, `pipe`, and `wait`. It builds pipelines by arranging file descriptors. It has no session, no `setpgid`, and no controlling-terminal foreground. That is enough to see why the descriptor half of section 5 exists. Linux adds the group and session objects so a terminal can name a job. Read the xv6 `sh` you have for the actual loop, and keep today's line numbers attached to `c9b2385d6793`.

---

8. What the wrapper is allowed to touch

Run `../challenges/day-006-shell-job-control.sh` from the Week 10 directory. It resolves the canonical project, builds with the project's Makefile, and runs without root. It prints this process's PID, parent, process group, session, and whether stdin is a terminal. It runs the fixed fork/exec/wait status mode. A short-lived helper creates one child, the child calls `setpgid(0, 0)`, and the parent checks the report and waits. Every descendant is reaped.

The run leaves `setsid` and `tcsetpgrp` uncalled. It sends no job-control signal. It implements no interactive read loop. Those are the operations that would move the IDE terminal's foreground group. Predict, before the run, that the status child decodes as exit 7 and that the helper child is a process-group leader. Treat the terminal answer as a property of the environment that started the wrapper.

---

9. Mastery: Explain · Draw · Observe · Build

The wrapper is `../challenges/day-006-shell-job-control.sh`. A passing run shows ids and one child group. The account of why a terminal needs a PGID is still yours. These notes leave mastery open.

Explain

1. What question does PGID answer that PID does not?
2. Why does `setpgid` consult `PF_FORKNOEXEC` on the parent's call?
3. What two pid types does `set_special_pids` retarget, and what does `proc_clear_tty` drop?
4. How is an orphaned process group different from Day 4's orphan process?
5. Why does a shell call `setpgid` in both the parent and the child?
6. Why is this mechanism the same C code on x86-64, AArch64, and RISC-V?

Draw

Draw a session with one foreground group of two pipeline members and one background group. Label SID, PGIDs, and the terminal's foreground id. Then draw the single-command trace from section 6 up through `exec` and `wait`, and mark the step the lab actually performs and the step it leaves to an interactive shell.

Observe

Run the wrapper. Record PID, PPID, PGID, SID, the stdin terminal result, the decoded status 7, and the helper's PID-equals-PGID result. Record `uname -r`. If you quote `setpgid` or `ksys_setsid`, name the tree.

Build

Write a paragraph that places one pipe, two children, and one PGID into the loop from section 5, including which descriptors each process closes. Keep it on paper or in the notebook. The wrapper stays the bounded run: one status child, one group-leader child, every child waited for.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Shell Job Control

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-006-shell-job-control.sh
```

- View lab source on GitHub

Day 7 — `/proc`, strace, GDB, and the Comparison

Target time: approximately 3 hours

> The kernel objects from this week are real, and user code cannot see them by reading its own variables. `/proc`, strace, and GDB are three ways to ask the kernel to describe a task. Each one stops at a different layer, and each one is evidence only after you have named the kernel, the binary, and the question.

Days 1–6 followed a task from identity through `fork`, `exec`, the exit word, signal delivery, and process-group membership. Today those mechanisms become a report. The report says which fact came from a generated procfs file, which came from a ptrace stop, which came from an ELF symbol, and which came from a checked assertion in the lab program. A tool transcript with no version and no scope is a story about a different machine.

Line-level claims below are from tree `c9b2385d6793`, a 5.10.134-based kernel. Before you repeat a line number, record `uname -r` and the tree id you actually opened. Names move. A function that is `proc_pid_status` here may be split or renamed in a later release, and the string it prints can gain fields. The man page for procfs is the application contract. This file is one kernel's generator.

---

1. Why the view has to be generated

A task's exit state, real parent, blocked signals, and session id live in kernel memory, in objects the C abstract machine of the process does not name. Week 2's process can see its own globals, its stack, and the bytes it loads and stores. It cannot see `exit_state`. The kernel therefore builds a text file on demand when someone reads `/proc/PID/status` or `/proc/PID/stat`.

That read is a syscall like any other. The vfs path from Week 9 resolves the procfs inode, and the inode's operations call a function that `seq_printf`s fields out of `task_struct`. Two reads are two generations. A task can change state between them. The file is a projection taken while the kernel held the locks that function takes, and it is a projection of the reader's pid namespace and credentials. It is fresh text, produced for that `read`.

`fs/proc/base.c` in this tree publishes the name. The task directory contains an entry of the form `ONE("status", S_IRUGO, proc_pid_status)` (the occurrences are at lines 3201 and 3545). Opening `status` and reading it runs `proc_pid_status`.

---

2. The state letter is an index into a table

`fs/proc/array.c`, tree `c9b2385d6793`, lines 124–150:

```
/*
 * The task state array is a strange "bitmap" of
 * reasons to sleep. Thus "running" is zero, and
 * you can test for combinations of others with
 * simple bit tests.
 */
static const char * const task_state_array[] = {

    /* states in TASK_REPORT: */
    "R (running)",      /* 0x00 */
    "S (sleeping)",     /* 0x01 */
    "D (disk sleep)",   /* 0x02 */
    "T (stopped)",      /* 0x04 */
    "t (tracing stop)", /* 0x08 */
    "X (dead)",     /* 0x10 */
    "Z (zombie)",       /* 0x20 */
    "P (parked)",       /* 0x40 */

    /* states beyond TASK_REPORT: */
    "I (idle)",     /* 0x80 */
};

static inline const char *get_task_state(struct task_struct *tsk)
{
    BUILD_BUG_ON(1 + ilog2(TASK_REPORT_MAX) != ARRAY_SIZE(task_state_array));
    return task_state_array[task_state_index(tsk)];
}
```

The comment values match the flags in `include/linux/sched.h` in this tree: `__TASK_TRACED` is `0x0008`, `EXIT_DEAD` is `0x0010`, `EXIT_ZOMBIE` is `0x0020`. Day 4's assignment `tsk->exit_state = EXIT_ZOMBIE` is why a later read can print `Z (zombie)`. A ptrace stop, below, is why the same table can print `t (tracing stop)`. The letter in `/proc/PID/stat` is the first character of that string. `do_task_stat` takes it with `state = *get_task_state(task)` at line 460. The long form in `status` prints the whole string.

`get_task_state` indexes the table. It does not run the program, sample the user instruction pointer, or reconstruct a backtrace. The scheduler and the exit path already stored a state. Procfs translates the stored value into text.

---

3. `proc_pid_status` walks the task and prints

`fs/proc/array.c`, `proc_pid_status`, lines 408–432:

```
int proc_pid_status(struct seq_file *m, struct pid_namespace *ns,
            struct pid *pid, struct task_struct *task)
{
    struct mm_struct *mm = get_task_mm(task);

    seq_puts(m, "Name:\t");
    proc_task_name(m, task, true);
    seq_putc(m, '\n');

    task_state(m, ns, pid, task);

    if (mm) {
        task_mem(m, mm);
        task_core_dumping(m, mm);
        task_thp_status(m, mm);
        mmput(mm);
    }
    task_sig(m, task);
    task_cap(m, task);
    task_seccomp(m, task);
    task_cpus_allowed(m, task);
    cpuset_task_status_allowed(m, task);
    task_context_switch_counts(m, task);
    return 0;
}
```

The function is a list of projections. Name, then `task_state`, then memory if the task still has an `mm`, then signal sets, capabilities, seccomp, and cpuset. A zombie has already passed `exit_mm` on Day 4, so the `if (mm)` body is skipped and the file still carries the state line from `task_state`. That is why `Z` can appear in a file that has no user address space left to describe.

`task_state` is where Pid and PPid are computed. Lines 163–192:

```
    rcu_read_lock();
    ppid = pid_alive(p) ?
        task_tgid_nr_ns(rcu_dereference(p->real_parent), ns) : 0;

    tracer = ptrace_parent(p);
    if (tracer)
        tpid = task_pid_nr_ns(tracer, ns);

    tgid = task_tgid_nr_ns(p, ns);
    ngid = task_numa_group_id(p);
    cred = get_task_cred(p);

    task_lock(p);
    if (p->fs)
        umask = p->fs->umask;
    if (p->files)
        max_fds = files_fdtable(p->files)->max_fds;
    task_unlock(p);
    rcu_read_unlock();

    if (umask >= 0)
        seq_printf(m, "Umask:\t%#04o\n", umask);
    seq_puts(m, "State:\t");
    seq_puts(m, get_task_state(p));

    seq_put_decimal_ull(m, "\nTgid:\t", tgid);
    seq_put_decimal_ull(m, "\nNgid:\t", ngid);
    seq_put_decimal_ull(m, "\nPid:\t", pid_nr_ns(pid, ns));
    seq_put_decimal_ull(m, "\nPPid:\t", ppid);
    seq_put_decimal_ull(m, "\nTracerPid:\t", tpid);
```

`PPid` is `real_parent` translated into the reader's pid namespace. Day 4's reparenting changes `real_parent`, and the next `status` read prints the new number. `TracerPid` is the ptracer when one is attached, which is a different parent pointer from `real_parent`. Day 6's session and process group show up in the `NSpid` / `NSpgid` / `NSsid` lines this function prints a little later, under `CONFIG_PID_NS` (lines 212–224). The numbers are namespace-relative. A container and its host can print different pids for one task.

`task_sig`, called from `proc_pid_status`, prints the pending, shared pending, blocked, ignored, and caught sets through `render_sigset_t` (the calls are at lines 298–302). Day 5's three facts — pending, blocked, disposition — have a text form here. A blocked `SIGUSR1` that has been posted can show up in `SigPnd` or `ShdPnd` while `SigBlk` contains the same bit. That is the pending-mode experiment, seen from procfs in the same form `sigpending` reports.

The lab's `proc_lab` reads its own `status`, the first line of `stat`, and the `exe` symlink. `exe` is a symlink to the ELF file Day 3 mapped. A zombie has dropped that mapping. A live process's `readlink` of `/proc/self/exe` answers "which file was exec'd," which can differ from the path the shell typed if the file was reached through a link. The lab checks the open, the read, the close, and the `NUL` terminator. A deleted-file suffix or an inaccessible link is a property of this environment when it happens.

`/proc/PID/stat` is a single line and a different function, `do_task_stat`. It is picky about the command field: the name sits in parentheses and can contain spaces, so a parser finds the last `)`. The lab's own child has a controlled name. The instruction pointer in that line is deliberately uninformative for a running task:

```
    state = *get_task_state(task);
    vsize = eip = esp = 0;
    permitted = ptrace_may_access(task, PTRACE_MODE_READ_FSCREDS | PTRACE_MODE_NOAUDIT);
    mm = get_task_mm(task);
    if (mm) {
        vsize = task_vsize(mm);
        /*
         * esp and eip are intentionally zeroed out.  There is no
         * non-racy way to read them without freezing the task.
         * Programs that need reliable values can use ptrace(2).
         *
         * The only exception is if the task is core dumping because
         * a program is not able to use ptrace(2) in that case. It is
         * safe because the task has stopped executing permanently.
         */
```

`fs/proc/array.c`, `do_task_stat`, lines 460–474. The comment is the bridge to the next section. User `rip` / `pc` / `epc` and the user stack pointer change whenever the task runs. A procfs read would sample them without stopping the task, and the sample would be a race. The kernel prints zeros and tells you to stop the task with `ptrace` if you need the registers. The core-dump exception in the comment is the case where the task has already stopped permanently, so the saved registers stay still.

---

4. A ptrace stop is a task state

strace and GDB are both tracers. They attach with `ptrace`, and the kernel parks the tracee in `TASK_TRACED` until the tracer says to continue. That park is the same kind of kernel state as `EXIT_ZOMBIE`: a value in the task, visible to `wait` and to procfs, with no user instruction retiring while it holds.

`kernel/signal.c`, `ptrace_stop`, tree `c9b2385d6793`, lines 2133–2160:

```
    /*
     * schedule() will not sleep if there is a pending signal that
     * can awaken the task.
     */
    set_special_state(TASK_TRACED);

    /*
     * We're committing to trapping.  TRACED should be visible before
     * TRAPPING is cleared; otherwise, the tracer might fail do_wait().
     * Also, transition to TRACED and updates to ->jobctl should be
     * atomic with respect to siglock and should be done after the arch
     * hook as siglock is released and regrabbed across it.
     *
     *     TRACER                   TRACEE
     *
     *     ptrace_attach()
     * [L]   wait_on_bit(JOBCTL_TRAPPING)   [S] set_special_state(TRACED)
     *     do_wait()
     *       set_current_state()                smp_wmb();
     *       ptrace_do_wait()
     *         wait_task_stopped()
     *           task_stopped_code()
     * [L]         task_is_traced()     [S] task_clear_jobctl_trapping();
     */
    smp_wmb();

    current->last_siginfo = info;
    current->exit_code = exit_code;
```

`set_special_state(TASK_TRACED)` is the store. `TASK_TRACED` includes `__TASK_TRACED`, the bit `get_task_state` maps to `t (tracing stop)`. The comment's two-column picture is the memory-ordering reason for `smp_wmb`: the tracer's `wait` must observe `TRACED` before it trusts the stop. `last_siginfo` and `exit_code` hold what the tracer can read back about why the stop happened. Day 4 used `exit_code` for a terminating task. Here the same field carries the stop code while the task is alive and frozen for the tracer.

`get_signal` calls `ptrace_signal` before it honors a caught disposition (Day 5, around line 2664). A tracer can therefore stop on a signal, rewrite it, or swallow it, and the tracee stays in `TASK_TRACED` across that stop. Syscall tracing uses the same state at syscall entry and exit: the architecture entry path notices the trace flag, and the task stops where the tracer can read the syscall number out of the ABI register.

That register is architectural. On x86-64 the number is in `orig_ax` after a `syscall` instruction. On AArch64 it is in the syscall-number register the entry code saved, conventionally `x8`. On RISC-V it is `a7`. strace's table from number to name is a per-ABI table. The stop itself — `TASK_TRACED`, `wait`, continue — is the shared kernel mechanism. Day 5's signal frame was architectural because the handler call is architectural. A ptrace stop is kernel task state, and the registers the tracer then reads are architectural.

While the tracee is stopped, its user registers sit in the kernel's `pt_regs` for that task, saved by the trap or syscall entry the way Week 8 saved registers around `int 0x80` and the way ToyOS saves them for `iretq`. The tracer issues `ptrace` requests to copy those registers out. When the tracer continues the task, the kernel returns to user mode through the normal

exception-return path, and the user instruction stream resumes. Stopping changed the timing. A race that appears under strace or GDB has been sampled under a scheduler the tracer perturbed.

---

5. strace is that stop, printed as a syscall list

strace attaches, asks to stop at syscall entry and exit, and prints a decoded line: name, arguments, return value. With `-f` it follows children, which is how a fork/exec/wait trace stays in one transcript. The line is the tracer's formatting of register values and of the kernel's return code. It is evidence about which syscalls the dynamic loader and libc issued. It is a decoded view, and the spelling depends on the ABI and on the strace version's tables.

For this week's status mode, a useful trace shows the create call (`clone` or `clone3`, whichever libc used), the `execve` of the fixed program, and the parent's `wait4`. An `execve` that returns is a failure in the old image. An `execve` that is followed by the new program's own syscalls, and then by an exit of 7, is the success path Day 4 decoded from the wait word. Match the wait status with `WIFEXITED` in the program. The trace shows the raw return; the macros are still how C reads it.

The wrapper traces only the fixed status mode, writes the log under its temporary report directory, and extracts a few landmarks. If strace is absent, the honest line is a skip. A skipped tool is a limit on the report. Inventing a syscall line would attach the report to a machine you did not run.

Attaching is itself a ptrace stop. The child runs more slowly, and the order of two runnable tasks can differ from an untraced run. Use the trace to identify the boundary — before `fork`, before `exec`, inside the new image, inside a signal, inside `wait` — and keep the timing claim inside what one trace can support.

---

6. GDB is the same stop, plus symbols

GDB attaches or launches through ptrace as well. The extra ingredient is the ELF symbol table and DWARF line table Week 3 produced at link time. A breakpoint at `main` is an address the linker assigned to that symbol, patched into the process so that execution stops and the kernel reports a ptrace stop. The command file `gdb/week10.gdb` breaks at `main`, runs, and continues. In batch mode against `proc_lab`, a successful capture is: this binary, this symbol, this stop, then the program's own checked marker.

`main` is a C function. The first user instruction after `exec` is the ELF entry or the dynamic linker, as Week 3's loading lesson followed. A breakpoint at `main` fires after that startup. Registers at the stop are the `pt_regs` from section 4, shown with names GDB took from the ABI and from the debug info. Locals appear when DWARF describes them. A stripped binary can still have a function symbol and lack the local names. Say which of those you actually had.

Fork needs an explicit follow-fork setting, because the new task is a different `task_struct` and GDB is attached to one of them. `exec` replaces mappings, so the symbols GDB holds have to be reread for the new image. A signal can be passed, stopped, or swallowed according to GDB's handle settings, which is `ptrace_signal` from user space. The command file is evidence when GDB ran. A file that was not executed is a recipe, and the report says the recipe was not run.

Predict the stop before you look. "Break at `main`, print one argument, continue, expect the lab marker" is a prediction. A transcript that confirms it is an observation. A transcript that surprises you names the first contradicted fact and stops there, which is a better debugging step than collecting every register.

---

7. Version discipline

Write these down before a line-number claim:

```
uname -r                        the kernel that produced /proc and ptrace
tree id c9b2385d6793            the 5.10.134-based sources these notes quote
config / distributor patches    if you discovered the running tree differs
architecture                    x86-64, AArch64, or RISC-V, for syscall names
compiler, libc, strace, GDB     when a tool's decoding is part of the claim
binary hashes                   the ELF files the lab actually exec'd
```

The running kernel and the tree you are reading can differ. Fedora, the lab VM, and this Anolis-based tree can carry the same mechanism under shifted line numbers. When they differ, describe the mechanism at the level that survived — `EXIT_ZOMBIE` is a retained exit word, `wait4` copies it out, a signal frame is architectural — and attach line numbers only to the tree you opened. The notes in Days 4–6 did that for `c9b2385d6793`. Your report does it for the machine that printed `uname -r`.

Syscall numbers, `/proc` field order, and GDB's default follow-fork behavior are versioned. The stable questions are the ones the lab checks: decoded wait status, `Z` then a reap, pending-then-delivered, a child whose PGID equals its PID, and the injected errno values.

---

8. ToyOS, xv6, and this Linux tree

The comparison is about who holds the exit word, who can stop a task, and what a debugger can see. It is three concrete designs.

ToyOS, Weeks 6–9. One user address space. A fault uses an IDT stub and, for the vectors ToyOS assigns, an IST stack, and `iretq` resumes the interrupted context (Week 6). The observable markers are explicit: a serial line, a panic string, a GDB remote stop against the ELF symbols. Week 8 adds a single user image and an `int 0x80` syscall path, including a pipe, in that one address space (Week 8). There is no second task, so there is no `real_parent`, no `EXIT_ZOMBIE`, no `sigaction` table, and no process group. The frame that resumes a context is the trap frame. Linux grew a user signal frame and a `rt_sigreturn` on top of the same idea, and a task list so that a second task can wait.

xv6. A small teaching kernel with a process table. `fork` allocates a slot and duplicates the caller's user memory. `exec` replaces the image in that slot and keeps the pid. `exit` stores a status in the slot and marks it a zombie. `wait` in the parent copies the status out and frees the slot. An orphan's parent pointer is moved to the init process. The shell is a loop of `fork`, `exec`, `pipe`, and `wait`, which is the descriptor half of Day 6 without sessions or a foreground group. Signals, in the usual base system, are a small kill check on the way back from a trap, far from Linux's queued `siginfo` and architecture frame. Read the xv6 revision you have for the real function names. The idea is the comparison. Pasting a textbook page would freeze someone else's line numbers into this report.

This Linux tree. `task_struct` is the slot, with sharing flags so threads can share an `mm` and a file table. `do_exit` stores `exit_code` and `exit_notify` publishes `EXIT_ZOMBIE` until `wait_task_zombie` copies it through `wait4`. `find_new_reaper` prefers a subreaper over the namespace init. `__send_signal` posts a pending bit, `get_signal` chooses a disposition, and the architecture file builds the frame. `setpgid` and `ksys_setsid` maintain PGID and SID. `proc_pid_status` prints the result. `ptrace_stop` freezes a live task in `TASK_TRACED` so strace and GDB can read `pt_regs`. Namespaces, credentials, and thread groups sit on the same objects. The extra machinery is more state with an owner, which is why the lab reaps every child and limits signals to the process family it created.

```
question            ToyOS                   xv6                     Linux c9b2385d6793
exit word           the one task ends       slot status, zombie     exit_code, EXIT_ZOMBIE
who collects it     the run ends            wait in the parent      wait4 / subreaper
image replace       the loader you wrote    exec in the slot        execve + ELF handler
stop a live task    GDB / explicit marker   debugger on the kernel  TASK_TRACED, ptrace
user resume         iretq                   trap return             exception return,
                                                                    rt_sigreturn for handlers
job name            one context             parent/child            PGID, SID, foreground
text view           serial log              console                 procfs generated
```

One invariant per row is enough. "Who owns the exit status" separates all three. "Where a pending asynchronous event sits" separates ToyOS's trap frame from Linux's pending bitmap. "What a debugger stops" is a GDB remote protocol on ToyOS and a `TASK_TRACED` task on Linux.

---

9. Execution trace: one report

This is the shape of `../challenges/day-007-debug-report.sh`. The wrapper builds the canonical programs, runs the checked modes, and writes a temporary directory. It prints the path. It runs without root and traces only lab processes.

```
record uname -r, date, compiler, and tool versions
hash the three canonical binaries
read ELF class / machine / type

proc_lab
    open /proc/self/status
        proc_pid_status → task_state → State, Pid, PPid
    open /proc/self/stat
        do_task_stat → one state letter, eip/esp left 0
    readlink /proc/self/exe
        the ELF file this image came from

status mode
    fork, exec, wait, decode exit 7
    optional strace -f
        clone-family, execve, wait4
        or the line "strace unavailable"

optional gdb -batch -x gdb/week10.gdb --args proc_lab
    ptrace stop at main
    continue
    program marker
    or the diagnostic if ptrace policy refused the attach

injected failures
    exec of a missing path → child _exit(127) → parent decodes it
    second waitpid → ECHILD
    sigaction(SIGKILL) → EINVAL
    kill(pid that names nothing, 0) → ESRCH

every child waited for
WEEK10 PASS when the checked invariants hold
```

`WEEK10 PASS` means those invariants held on this run. It is a property of the programs, the kernel, and the tools that were present. It leaves the comparison section and the limits for you to write in your own words.

The injected failures are specified detectors. The missing path is `ENOENT` in the child, mapped to exit 127; another exec error maps to 126. The parent checks 127, then waits again and requires `ECHILD`, which is Day 4's consumed-status observation. `SIGKILL` keeps its kernel disposition, so `sigaction` fails with `EINVAL`. Signal zero posts nothing and still reports `ESRCH` when the pid names no task. The lab picks a pid outside the ordinary range for that probe. Those four results are assertions. They are not a search of every errno.

---

10. What the wrapper is allowed to touch

Run the wrapper from the Week 10 directory. It uses the project Makefile, creates only descendants, reaps them, and sends signals only where the signal-lab modes already do: to the lab process or its children. Optional tools stay optional. A ptrace policy that refuses GDB belongs in the report as a refusal, with the diagnostic the tool printed.

Predict `WEEK10 PASS`, three hashes, the procfs fields for the lab process itself, the decoded statuses, and either a real strace/GDB transcript or an explicit skip. Then write one sentence of comparison for the exit word, in the form section 8 used, tied to the sources you opened.

---

11. Mastery: Explain · Draw · Observe · Build

The wrapper is `../challenges/day-007-debug-report.sh`. A passing run collects evidence. Mastery is the report you can defend, and these notes do not record it as complete.

Explain

1. Why does `proc_pid_status` run at read time, and what lock or RCU section does `task_state` use while it reads `real_parent`?
2. Why can `status` show `Z (zombie)` after `exit_mm` has dropped the address space?
3. Why does `do_task_stat` leave `eip` and `esp` at zero for a running task, and what mechanism does its comment name instead?
4. What task state is a strace stop, and which part of the printed line is architectural?
5. Why is a breakpoint at `main` later than the first user instruction?
6. What does ToyOS lack that makes `wait4` unnecessary there, and what does xv6's process slot store that Linux stores in `exit_code`?

Draw

Draw the path from `read(/proc/self/status)` to the `State:` line, with `proc_pid_status`, `task_state`, and `get_task_state` labeled and the tree id in the corner. Beside it, draw a tracer and a tracee at a syscall stop: `TASK_TRACED`, `pt_regs`, and the ABI register that holds the syscall number on the architecture you ran.

Observe

Run the wrapper. Keep `uname -r`, the tree id if you opened the sources, the three hashes, the procfs excerpt, the status decode, and the strace/GDB result or the skip line. Quote a kernel line only from a tree you identified.

Build

Write the comparison in your report with one invariant per row, using section 8's questions. Name the ToyOS marker you would actually look at (serial line or GDB), the xv6 idea in your own words, and the Linux function from this tree that implements the same idea. Leave the wrapper's temporary report path in the write-up so the evidence can be found again.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Debug Report

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-007-debug-report.sh
```

- View lab source on GitHub